Application Intelligence

Semantic Streams Augmentation

2026 Sebastián Samaruga. https://sebxama.blogspot.com

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Table of Contents

Summary
- Infer Intra and Inter Application Use Cases
- Features Exposing the Unified Systems Facade

Base Model
- Resources
- Occurrences
- Statements
- Context, Subject, Predicate, Object Resources
  - Context
  - Subject
  - Predicate
  - Object
- Kinds
  - Context Kinds
  - Subject Kinds
  - Predicate Kinds
  - Object Kinds
  - Kinds as Extensional Functional Definitions

Base Model Representations
- RDF Model Representation
- Sets Based Model Representation
- ISO TMRM Model Representation
- FCA Context Lattices Model Representation
- XML Model Representation
- Representations Summary

Execution Model: Augmentation Pipeline Agents
- Homoiconic (code as data) Approach
  - Streams Augmentation Services Agents Model
    - Fold
    - Unfold
    - Merge
- Messaging Infrastructure
  - Messaging Format Representation: TreeGraph

- Streaming Agents Augmentation Pipeline
  - Aggregation Service Agent
    - Aggregation Agent Domain Model
    - Aggregation Agent Merge / Fold / Unfold
  - Alignment Service Agent
    - Alignment Agent Domain Model
    - Alignment Agent Merge / Fold / Unfold
  - Activation Service Agent
    - Activation Agent Domain Model
    - Activation Agent Merge / Fold / Unfold
  - Helper Services
    - Registry Service
    - Naming Service
    - Index Service
- State / IO Functional Addressing Features
- Datasources
- Dynamic API Endpoints
  - Dynamic API Endpoints Integration
    - Monadic Context & Interaction Pipeline
    - Protocol Execution & Resource Traversal
    - Generic Dynamic API Endpoints Browser / Designer
  - Dynamic API Endpoints API Architecture & Monadic Workflow
    - Monadic Execution Model: State and IO Monads
    - Activation Statements Transformation Flow
    - Conversational Session & Resource Navigation Pattern
  - Dynamic API Endpoints Summary
    - Resource Representation and Navigational State
    - State Monad: API Navigation and Augmentation State
    - IO Monad: API Representation ↔ Activation Statements
    - Generic Contexts / Roles / Actors / Interactions Resources
    - Browsing a Use Case as REST Resource Navigation
    - Resource Rendering from Activation Production Statements
    - Executing an Interaction
    - Parent, Child and Placeholder Resources
    - API Facade and Augmentation Pipeline Feedback
    - Generic Dynamic API Protocol
  - Generic Dynamic API Endpoints Browser / Designer
- Implementation Techniques
  - Leverage TMRM (ISO Topic Maps Reference Model)
  - Leverage Dimensional Features
  - Leverage Formal Grammars
  - Leverage RDF, RDFS, OWL and SPARQL
  - Leverage Sets Model
  - Leverage ML and LLMs

- Leverage MCP (Model Context Protocol)
- Leverage FCA
  - FCA-based Relational Schema Inference
- Leverage CPPE
  - Algebraic Semantic Embeddings (CPPE):
    - Prime Numbers Identifiers:
    - Prime ID Embeddings:
    - CPPE. FCA-based Embeddings: A Deterministic Approach:
      - Relational Context Vectors
      - Schema Archetypes:
      - Subsumption:
      - Property Chains
      - Querying and Traversal by Numerical Properties:
      - Graph Pipeline Specification: CPPE Algebra for Pipeline Stream Actions:
      - 1. Algebraic Definitions
        - The Prime Product Rule:
      - 2. Pipeline Stream Operators (The CPPE Monad)
        - A. The "Contains" Operator (Sub-graph Filtering):
        - B. The "Intersection" Operator (Aggregation):
        - C. The "Axis Shift" Operator (Alignment):
      - 3. Pipeline Stages Revisited via CPPE:
        - 3.1 Aggregation (Algebraic Type Inference):
        - 3.2 Alignment (Vector/Prime Space Prediction):
        - 3.3 Activation (State Transition Product):
- Leverage W3C DIDs (Distributed Resource Identifiers)
  - The resulting principle
- Techniques Summary

Appendix: Enterprise Scenario: Cross-System Order-to-Delivery Integration

- 1. Raw Source Data Streams (Datasource Ingestion)
- 2. Stage 1: Aggregation Service Agent (Data Layer - Type & Role Inference)
  - Processing & Inference
  - Production Output (Property Quad Statements)
- 3. Stage 2: Alignment Service Agent (Information Layer - Schema Matching & Identity Resolution)
  - Processing & Inference
  - Production Output (Rule Quad Statements)
- 4. Stage 3: Activation Service Agent (Knowledge Layer - Use Cases & Behaviors)
  - Processing & Inference
  - Production Output (Executable Context Quad Statements)
- 5. Unified Contexts & Interactions API (Dynamic Endpoint Execution)
  - Architectural Summary Table
  - Execution Flow Example

Appendix: Use Case Discovery and Unified Facade

- Use Case Discovery Pipeline

- Intra-Application Use Case Discovery
- Inter-Application Use Case Discovery
- Use Case as an Inferred Semantic Contract
- Use Case Discovery from Dimensional and State Structures
- Composite Interactions
- Unified Read Facade and Unified Interaction Facade
  - Unified Resource Facade
  - Unified Interaction Facade
- Traceability from Facade Interaction to Datasources
- Datasource Synchronization
- Facade Generation Principle
- Relationship to Implementation Techniques
- Resulting architecture
- 1. Intra-application: Employee lifecycle facade
  - What Aggregation discovers
  - What Alignment discovers
  - What Activation exposes
- 2. Intra-application: Order fulfillment facade
  - Aggregation
  - Alignment
  - Activation
- 3. Inter-application: Customer 360 facade
  - Aggregation
  - Alignment
  - Unified facade
- 4. Inter-application: University ↔ employment ecosystem
  - Aggregation
  - Alignment
  - Activation
- 5. Inter-application: Product-to-service lifecycle
  - Alignment features
  - Unified facade
- 6. Inter-application: Lead → Customer → Contract → Service
  - Aggregation
  - Alignment
  - Activation
- 7. Inter-application: Finance reconciliation
  - Alignment
  - Activation
- 8. A more interesting class: inferred "composite" interactions
  - How the pipeline would infer these use cases
  - What the unified facade could actually expose

Appendix: ChatGPT Conversation

Appendix: Gemini Conversation

Summary

This document describes a semantic RDF based data integration and intelligence framework proposal. By means of an Aggregation, Alignment and Activation Augmentation pipeline graphs stream processing in the form of Context, Subject, Predicate, Object Statements inputs: Infer graph Resource types, Occurrence Roles (Aggregation). Merge / cluster equivalent entities, complete missing attributes / links / relationships (Alignment). Infer Use Case types: Contexts / Roles, and Use Case executions: Interactions / Actors (Activation).

From raw source graph data in the form of RDF Statements, from any backend / datasource capable of being exported into an (RDF) graph, obtain an executable representation of the underlying models: integrated applications backends / datasources graph representations. Intra and inter integrated applications Use Cases will be discovered and exposed through an unified Contexts and Interactions API Facade.

The underlying integrated applications backends / datasources are kept in sync with this framework inferences and Context(s) Interaction(s) data and state flows and transforms. The integrated applications keep working, reflecting changes made via these integration mechanisms, while this framework behaves as an interaction integration “agent”, capable of viewing and exposing an unified integration view of all the underlying integrated applications for further development of inter / intra application integration use cases.

A unified integration facade is generated from the semantic Contexts and Interactions inferred over the integrated datasource graph. Resource views expose the resulting integrated state; Interaction endpoints expose executable Contexts and state transitions; and each facade operation remains connected to the underlying Statements, Rules and application resources from which it was inferred.

The integrated facade is not a canonical CRUD API generated from a merged database schema. It is a semantic interaction facade generated from inferred Resources, Kinds, Relationships, Contexts, Roles, Actors and Interactions. Resource views provide unified read access, while Activation provides contextual operations whose execution may span multiple underlying applications.

This can be regarded as “reverse engineering” of business applications rendering them in an executable semantic model, which can run in parallel with original business applications, thus providing means for outlining potential inter-applications integration use cases.

Infer Intra and Inter Application Use Cases

- **Intra-Application Schema and Type Discovery (Data Aggregation)**: Raw RDF graph statements exported from individual backend datasources can be processed to infer hidden object types, property definitions, and relational schemas. For example, analyzing raw triple statements like (:dev1, :worksOn, :projA) allows the framework to

infer a structural DeveloperWorksOnProject schema with defined domain (Developer) and range (Project) types.

- **Inter-Application Entity and Identity Resolution (Information Alignment)**: Equivalent entities and relationships across disparate backend streams can be merged, filling in missing attributes or cross-system links. This includes discovering structural analogies across different domain contexts, such as recognizing that Employee is to Employer in a corporate backend as Student is to School in an academic backend.
- **State Transition and Event Flow Tracking (Information Alignment)**: Sequential data updates within or across applications can be mapped into ordered state networks. For instance, tracking previous, current, and next (PCN) states allows systems to infer behavioral paths like (Single, Marriage, Married) or employment cycles like (Unemployed, Employment, Employee). This knowledge is dimensionally aggregated by means of Kind Dimensional Context Statements (ContextKinds) axes points resolution.
- **Cross-System DCI Contexts and Role Executions (Knowledge Activation)**: Inter-application interactions are mapped into executable Data-Context-Interaction (DCI) contexts. Multi-department operational scenarios—such as linking employee role definitions, position timelines, and departmental interactions—are unified into an executable context statement like (EmploymentContextCK, EmploymentInteractionSK, EmploymentPositionRolePK, EmploymentEmployeeActorOK).

- The framework infers Use Case types by modeling them as Contexts and Roles.
- The framework infers Use Case executions by modeling them as Interactions and Actors.
- By utilizing FCA / CPPE (deterministic), Machine Learning and LLMs, the system can estimate or infer state transitions and interaction outcomes directly from contextual information.
- Using Dimensional Features, the framework infers types and instances of state transition networks (Previous, Current, Next) based on events, enabling the system to recognize structural hierarchies and contextual behaviors.
- Through FCA (Formal Concept Analysis), the framework infers relational schemas and upper-level ontology structures from observed statement patterns, providing the semantic vocabulary for roles and interactions.

Features Exposing the Unified Systems Facade

- **Dynamic Contexts and Interactions API**: A HAL-like / Representational (Conversation) State Transfer (REST) discoverable API layer that renders user interfaces and execution boundaries directly over activated knowledge statements. It exposes both system states (nouns/properties) and role behaviors (verbs) to represent interactive workflows dynamically. The framework exposes HAL-like or REST discoverable behavior APIs over activated knowledge statements, rendering message-driven interfaces for context interaction instances and role executions.
- **Reactive Stream Augmentation Pipeline**: An asynchronous messaging pipeline (operating across Aggregation, Alignment, and Activation topics) that processes source data through merge, fold, and unfold stream operations. This keeps

underlying backends in sync with inferred context and state changes while exposing a unified integration view.

- **Model Context Protocol (MCP) Binding**: An interface layer that exposes discovered domain resources, alignment tools, and executable DCI interactions as context-aware tool endpoints for external AI agents and LLMs. This feature exposes the discovered semantic resources, contexts, and executable interactions as discoverable tool interfaces, allowing external AI agents and LLMs to dynamically inspect and trigger behaviors exposed by the unified API.
- **Distributed DID Registry and Endpoint Discovery**: A W3C Decentralized Identifier (DID) and DID Document registry layer that manages global resource identities, tracks data provenance, and advertises system service capabilities across administrative boundaries. DIDs establish cross-boundary identity and act as a distributed semantic registry, allowing the framework to advertise semantic business capabilities, synchronize graph states, and discover service endpoints across independently managed data sources.
- **CPPE Numerical Index and Query Engine**: An $O(1)$ arithmetic query mechanism that uses Contextual Prime Product Embeddings to execute sub-graph filtering (via prime divisibility/remainder), similarity matching (via Greatest Common Divisor), and instant type subsumption without performing expensive graph joins. This feature uses deterministic prime number mathematics for instant type checking and reactive stream filtering, triggering activations (like webhooks or UI changes) when a calculated state transition product matches a pre-calculated "Success Pattern".
- **ISO Topic Maps Reference Model (TMRM):** This feature provides the implementation-independent semantic substrate that handles deterministic identity resolution and topic merging, giving the pipeline a unified graph representation without losing the provenance or context of the underlying datasources.
- **Formal Grammars:** The system utilizes grammatical production rules to formalize stream processing, evaluating rules and parsing raw statements to dynamically generate prompts, contextual rules, and interaction executions.
- **RDF, RDFS, OWL, and SPARQL:** These standardized graph infrastructures guarantee universal interoperability, allowing the facade to ingest, match, and query raw data streams from any legacy backend capable of exporting an RDF graph.

Based on the provided specification, intra- and inter-application use cases are inferred primarily through the "Activation" (Knowledge) phase of the framework's Augmentation semantic stream processing pipeline, while multiple specific implementation techniques are leveraged to expose these as a unified systems facade.

The pipeline integrates source system data through three sequential stream processing stages:

- **Aggregation (Data Layer):** The framework ingests raw Context, Subject, Predicate, Object (CSPO) RDF Resource statements from the source systems and infers foundational types and roles. In the employment example, raw data is aggregated into production properties like EmploymentContextKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, and EmployerObjectKind.
- **Alignment (Information Layer):** The pipeline aligns these aggregated properties to establish dimensional rules, ontology matches, and links. It builds rule statements by

linking elements such as an EmploymentRelationshipCK (Context), an EmployeeRoleSK (Subject), and an EmployeePositionAtDatePK (Predicate).
- **Activation (Knowledge Layer):** The framework materializes the aligned rules into executable / executed use case roles, and interaction actors. For example, it translates the HR domain example rules into active contexts, producing statements that define the EmploymentCK (Context), EmploymentPositionRoleSK (Role), and EmploymentEmployeeActorPK (Actor).

By merging, folding, and unfolding these statement streams, the facade infers resource types, matches equivalent entities across disparate datasources, and completes missing attributes to expose the resulting behaviors as a unified, executable Activation domain model.

Base Model

```
class Resource
+ ID : URI
+ primeID : BigInteger
+ occurrences : Occurrence[]
+ apply(res: Resource[]) : Resource[]

class Occurrence extends Resource
+ player : Resource
+ context : Statement
+ role : Kind

class Statement extends Resource
+ context : Occurrence
+ subject : Occurrence
+ predicate : Occurrence
+ object : Occurrence

// Holds all a Resource’s Context Occurrences
class Context extends Resource

// Holds all a Resource’s Subject Occurrences
class Subject extends Resource

// Holds all a Resource’s Predicate Occurrences
class Predicate extends Resource

// Holds all a Resource’s Object Occurrences
class Object extends Resource
```

```
abstract class Kind<PlayerType extends Resource,
          AttributeType extends Resource,
          ValueType extends Resource> extends Resource
getPlayers : PlayerType[]
getAttributes(player : PlayerType) : AttributeType[]
getValues(player : PlayerType, attribute : AttributeType) : ValueType[]
getStatements() : Statement[]
getOccurrences() : Occurrence[]

class ContextKind extends Kind<SubjectKind, PredicateKind, ObjectKind>

class SubjectKind extends Kind<Subject, Predicate, Object>

class PredicateKind extends Kind<Predicate, Subject, Object>

class ObjectKind extends Kind<Object, Predicate, Subject>
```

Resources

Everything is a Resource identified by an URI, assigned a Prime Number ID, keeping track of its Occurrences and thus enabling the calculation of their CPPE (Prime Number Contextual Embedding) by means of the product of its Prime Number ID by the Prime Number IDs of the other Resources in their aggregated Occurrences contexts.

Occurrences

Participation of a Resource in an Statement occurrence playing a given role Kind in a given occurrence context (Statement) SPO role. Occurrence role Kinds are of their corresponding CSPO Kinds counterparts (ContextKind, SubjectKind, PredicateKind, ObjectKind) respectively.

Statements

Context, Subject, Predicate and Object Resource Occurrences (CSPO) tuples.

Context, Subject, Predicate, Object (CSPO) Statement Occurrences have a role (Kind) in their given Statement occurring Context.

Kinds: ContextKind, SubjectKind, PredicateKind, ObjectKind (CK, SK, PK, OK) are themselves reifiable as CSPOs (they are Resources) and can participate in a Statement Context as CSPO Occurrences Resources.

Statements Context ContextKind roles play the special function of aggregating other Statements role Kinds (SK, PK, OK) into a “dimensional” context kind. For example, the SPO

Kinds: (Employee, WorkingRelationship, Employer) can be aggregated into an “Employment” dimension.

Statements Reification:
Being a Statement an instance of Resource, it can play the Resource role in an Occurrence, being it a Statement Subject, Predicate or Object Occurrence player Resource.

Context, Subject, Predicate, Object Resources

Context, Subject, Predicate and Object are the positional roles of a Resource Occurrence instance in a Statement occurring context.

The role of a Resource Occurrence in a Statement is typed by its Statement Occurrence position and its Occurrence role type (Kind). A Resource can occur in more than one role Kind or occurrence position type (CSPO) in different Statements with its corresponding occurrence position role Kind type. The type of the role (Kind) of the Resource Occurrence is determined by the type of the position it plays in a given Statement (SK, PK, OK).

Context, Subject, Predicate and Object Resource classes are a “view” of those Resource Occurrences (URIs) typed by its Statement occurrence position role (CSPO).

Context

Resource (URI) whose Occurrences are the Statements in which this Resource URI occurs as a Statement Context.

Subject

Resource (URI) whose Occurrences are the Statements in which this Resource URI occurs as a Statement Subject.

Predicate

Resource (URI) whose Occurrences are the Statements in which this Resource URI occurs as a Statement Predicate.

Object

Resource (URI) whose Occurrences are the Statements in which this Resource URI occurs as a Statement Object.

Kinds

Kinds are a typing mechanism that performs aggregation by means of a Kind’s PlayerType parameter type instances sharing the same AttributeType parameter type instances (type aggregation) and sharing the corresponding AttributeValue parameter type instances for those AttributeType instances (state aggregation).

A Kind is defined by all of the Statements on which the Kind occurs as the Kind role of a corresponding CSPO Occurrence. So, Kinds can be viewed as a set of SPO Statements where the Subjects are the corresponding aggregated PlayerType instances, the Predicates are the corresponding aggregated AttributeType instances and the Objects are the corresponding aggregated AttributeValue instances.

Occurrences sharing common Attributes (type) and common Attribute Values (state) are aggregated into the same Kinds. They are Resources, reifiable as Subject, Predicate, Object Occurrence Resources playing the role of the Kind corresponding to its given Kind type aggregation (meta type) by their SPO position.

Kinds Aggregation:

Kinds aggregation is performed before the merge phase of an Augmentation pipeline layer. Resulting Kinds are merged into the same Kinds until Kinds identifying the same / equivalent instances are found. Kinds Player Type(s) Attribute Type(s) groups equivalent types. Kinds Attribute Type(s) Value Type(s) groups instances of a type in an equivalent state. Merge of Kinds determines whether Kinds are of the same type, same state and, if it is the case, whether two Kinds hold instances that are equivalent (same as).

The aggregated Kind role of a Resource Occurrence is to be assigned to the Statement Occurrence role type. The Occurrence (Statement) Context (ContextKind) Kind role is to be aggregated from its corresponding SubjectKind, PredicateKind and ObjectKind occurrences given those of the Statement SPO Occurrences role Kinds.

Kinds hierarchies, basic inheritance:

If a Subject attribute set is the superset of another Subject attribute set, it can be regarded as an “extension” of the first Subject type. TODO: Leverage this with other inference mechanisms (FCA, for example).

Kinds ordering:

A Subject “extended” (attributes subset) by another Subject attributes set can be regarded as being “before” the first Subject. This assumes the existence of a super type is necessary for the existence of a sub type.

Context Kinds

PlayerType: SubjectKind
AttributeType: PredicateKind
ValueType: ObjectKind

The Statement Context Occurrence ContextKind role Kind type aggregates the S, P, O occurrences role Kinds Player, Attribute and Values instead of the occurrences Resources. The aggregation process is the same as that of other Kind types: find / group by shared attributes (type aggregation), find / group by shared attribute values (state aggregation).

For example, for an Statement whose SPO Occurrences role Kinds are: (Employee, WorkingRelationship, Employee), a “dimensional” ContextKind “Employment” can be aggregated.

Subject Kinds

PlayerType: Subject
AttributeType: Predicate
ValueType: Object

Aggregates a set of Statements Occurrences whose Subjects share common Predicates and those Predicates share common Objects.

For example, for an Statement whose SPO Occurrences Resources are: (:Peter, :worksFor, :someEnterprise), a SubjectKind “Employee” / “SomeEnterpriseEmployee” can be aggregated.

Predicate Kinds

PlayerType: Predicate
AttributeType: Subject
ValueType: Object

Aggregates a set of Statements Occurrences whose Predicates share common Subjects and those Subjects share common Objects.

For example, for an Statement whose SPO Occurrences Resources are: (:Peter, :worksFor, :someEnterprise), a PredicateKind “WorkingRelationship” / “WorkingRelationshipAtSomeEnterprise” can be aggregated.

Object Kinds

PlayerType: Object
AttributeType: Predicate
ValueType: Subject

Aggregates a set of Statements Occurrences whose Objects share common Predicates and those Predicates share common Subjects.

For example, for an Statement whose SPO Occurrences Resources are: (:Peter, :worksFor, :someEnterprise), a ObjectKind “Employer” / “PeterEmployer” can be aggregated.

Kinds as Extensional Functional Definitions

Kind aggregated Statements can be regarded as the definition of a Function by extension. Their Statement Contexts Occurrence Resources are the Function name or signature. Their PlayerType(s) instances group their mappings by “context” and their AttributeType(s) and ValueType(s) instances are the Function domain / range corresponding value pairs. Note: Kinds are functional given their occurrences' dimensional context.

- CK: Context (SubjectKind), Domain (PredicateKind), Range (ObjectKind): aGivenContextKind(SK, PK): OK
- SK: Context (Subjects), Domain (Predicates), Range (Objects): aGivenSubjectKind(S, P): O
- PK: Context (Predicates), Domain (Subjects), Range (Objects): aGivenPredicateKind(P, S): O
- OK: Context (Objects), Domain (Predicate), Range (Subject): aGivenObjectKind(O, P): S

Base Model Representations

Base Model class hierarchy is represented by different meta model approaches (kept in sync with each other) for leveraging the most appropriate inference mechanisms.

The Base Model is represented simultaneously through several complementary meta-models. These representations are not competing alternatives; they are projections of the same `Resource → Occurrence → Statement → Kind` model, each optimized for particular forms of persistence, traversal, inference, comparison, aggregation, or stream processing.

Representations participate in the same **merge / fold / unfold** Augmentation pipeline (see “Execution Model”). One semantic model, multiple computational lenses.

RDF Model Representation

RDF Model Representation: This acts as the foundational raw data layer, utilizing a raw RDF Statements Triple (Quad) Store and a SPARQL Endpoint. It directly supports the objective by providing the standard graph representation and querying mechanism for the input and output streams of the semantic augmentation pipeline.

Role in the whole objective

- **Input:** natural representation for raw datasource statements.
- **Aggregation:** expose CSPO structure so occurrences can be grouped into `ContextKind`, `SubjectKind`, `PredicateKind`, and `ObjectKind`.
- **Alignment:** provide graph patterns, identity links, and relationship traversal.
- **Activation:** provide the graph substrate from which contexts, roles, actors, and interactions can be materialized.
- **Persistence/interoperability:** retain compatibility with RDF-oriented datasources and query infrastructure.

Sets Based Model Representation

Sets Based Model Representation: This representation organizes data into Universal Sets (Contexts, Subjects, Predicates, Objects) and Kinds Sets derived from their intersections

and unions. It leverages the framework's objective by treating data streams as mathematical collections, enabling deterministic operations for exact-match aggregations and deduplication.

Role in the whole objective

- **Aggregation:** unions/intersections provide deterministic grouping and exact-match type extraction.
- **Alignment:** set equality, intersection, inclusion, and difference provide basic matching and compatibility operations.
- **Activation:** sets can represent candidate states, possible transitions, or collections of executable structures before they are materialized as Statements.
- **Pipeline:** set operations can act as low-level primitives underneath higher-level merge/fold/unfold operations.

Contexts Set.

Subjects Set.

Predicates Set.

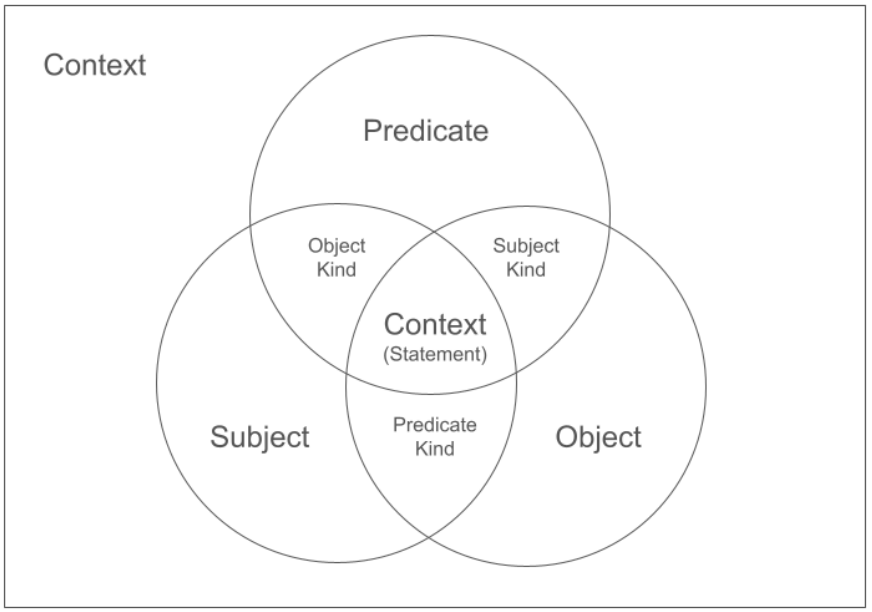
Objects Set.

Context (Statements) Set: (Subject Set, Predicate Set, Object Set intersection).

Subject Kinds Set (Predicates Set intersection with Objects Set).

Predicate Kinds Set (Subjects Set intersection with Objects Set).

Object Kinds Set (Subjects Set intersection with Predicates Set).



ISO TMRM Model Representation

ISO TMRM Model Representation: Operating as the underlying common metamodel, this maps the base entities onto TMRM subject proxies and properties. It supports the core objective by providing a subject-centric, implementation-independent foundation that drives deterministic identity resolution and topic merging across disparate data streams.

Role in the whole objective

- Provides the **subject-centric semantic substrate** for Resources and their contextual Occurrences.
- Preserves identity and context while navigating statements.
- Supports resource merging and graph traversal.
- Gives higher-level inference mechanisms a stable semantic model to operate on.

RDF provides the graph syntax; TMRM provides the semantic metamodel through which that graph can be manipulated as contextual subjects, occurrences, and statements.

FCA Context Lattices Model Representation

FCA Context Lattices Model Representation: This approach maps Contexts, Subjects, Predicates, and Objects into Formal Contexts. It advances the intelligence objective by allowing the system to build unified concept lattices, discovering implicit structural hierarchies, relational schemas, and classifications that are not immediately visible in isolated data streams.

The three FCA perspectives defined:

- **Subject as context:** predicates as objects, objects as attributes.
- **Predicate as context:** subjects as objects, objects as attributes.

- **Object as context:** subjects as objects, predicates as attributes.

Role in the whole objective

- **Aggregation:** infer recurring concepts, types, attributes, and shared properties.
- **Alignment:** discover higher-level conceptual equivalences, structural correspondences, domains/ranges, and relational schemas.
- **Activation:** use the inferred conceptual structures as the semantic basis for recognizing valid contexts and interactions.
- **Materialization:** convert lattice concepts back into Kinds / Statements so inference becomes persistent knowledge rather than an isolated analytical result.

XML Model Representation

TreeGraph for nested Statements structures and stream-processing messages.

XML Model Representation: This utilizes a TreeGraph format for handling nested Statements structures. It directly facilitates the execution model by defining the pipeline messages format, enabling the stream-oriented processing, folding, and unfolding of complex, layered message payloads across the augmentation agents.

Role in the whole objective

- Serialize a complete nested Statement/Kind structure into a single pipeline message.
- Preserve the contextual structure needed for fold/unfold operations.
- Support fan-in/fan-out asynchronous agent communication.
- Carry inferred production Statements between Aggregation, Alignment, and Activation agents.

RDF / TMRM / FCA / Sets are primarily semantic/computational representations; XML is primarily a structural transport representation for the stream-processing model.

Representations Summary

Representation	Primary strength	Aggregation	Alignment	Activation	Stream/Persistence role
RDF	Graph interchange & querying	Source statement analysis	Graph pattern matching	Knowledge graph substrate	Persistent/source graph

Sets	Exact structural algebra	Group/intersect	Equality/inclusion/difference	Candidate collections/state sets	Computational primitive
TMRM	Semantic metamodel & identity	Contextual resource classification	Identity/merge/traversal	Context/role graph substrate	Semantic persistence
FCA	Concept/lattice inference	Type & attribute inference	Concept/schema matching	Conceptual context recognition	Inference state/materialization
XML/TreeGraph	Nested serialization	Carry folded structures	Carry aligned structures	Carry executable structures	Pipeline messaging

The representations should remain semantically synchronized: an entity represented as a Resource/Occurrence/Statement in the Base Model can be projected into RDF triples/quads, set membership, TMRM subjects and occurrences, FCA contexts/concepts, or serialized TreeGraph messages without becoming a different semantic entity. Each projection exposes operations useful to a particular phase of the augmentation pipeline.

Execution Model: Augmentation Pipeline Agents

A sequential message driven execution model, the Augmentation Pipeline performs, by means of message exchange streams processing, the Semantic Streams Augmentation of raw RDF CSPO URIs Quads input Statements (from configurable Datasources, by means of appropriate system Statements) into executable models of knowledge gathered by means of (1) Aggregation, (2) Alignment and (3) Activation of Statements (messages) pipeline phases encoding corresponding layers domain models Statement types structures.

Augmentation Agents communicate with each other by means of a series of messaging endpoints where each layer consumes its input Statements and publishes their output Statements. The main operations a layer performs in a Statements stream are: type (Kinds) aggregation, Kinds merge, fold and unfolding of layers Statements.

Augmentation layers hold “state”, previously acquired knowledge given its past message exchange inputs in their corresponding level domain of abstraction. Each layer is considered to be a “lower level” representation of an upper / following layer level domain knowledge, from raw Statements to executable models.

Augmentation layers (Aggregation, Alignment, Activation) listen for Statements coming from an upper or lower layer.

Statements coming from an upper layer: TODO

Statements coming from a lower layer: TODO

This fan in / fan out approach is intended to be implemented in a reactive functional streams pipeline where each agent consumes and produces knowledge asynchronously and in parallel. An Agent output type Statement is another Agent source type Statement and an Agent output type Statement is the type of another Agent source Statement type. An Agent “accepts” an Statement if it is not known already or if it was further processed (updated) its context since the last consumption (or “acceptance” performance is no-op).

Endpoints dataflow is configured in this way:

Datasources <-> Aggregation <-> Alignment <-> Activation <-> Dynamic Endpoints API

Datasources
(raw Statements in / raw Statements out)
<->
(raw Statements in / raw Statements out)
Aggregation
(Aggregation Statements out / Alignment Statements in)
<->
(Aggregation Statements in / Alignment Statements out)
Alignment
(Alignment Statements out / Activation Statements in)
<->
(Alignment Statements in / Activation Statements out)
Activation
(Activation Statements out / Activation Statements in)
<->
(Activation Statements out / Activation Statements in)
Dynamic Endpoints API

TODO:

What layers consume / produce and how apply, merge, fold, unfold performs on each layer (layer / Graph methods):

LayerStatement : (Context, LayerStatement : parent, Attribute, Value)

Aggregation produces:
. (CK, S, P, O)
. Instance : (Class, Instance, Attribute, Value) : DOM
. Statement Resource URI encodes Instance value (URI). Statement Instance is Occurrence of parent Instance Statement.

. Until Instance definition (Classes). Instance Containment / Order (occurrence state / temporal? parts) hierarchies.

Alignment produces:
. (CK, SK, PK, OK)
. Fact : (Dimension, Fact / MeasurementValue, Axis, AxisValue) : Dimensional Alignment
. Facts table with dimensions tables FKs. Graph / Quad Statements encoding.
. Statement Resource URI encodes Fact value (URI). Statement Fact is Occurrence of parent Fact Statement.
. Until Fact definition (Dimension). Facts Containment / Order hierarchies.
. Ontology matching, links / attributes prediction: Same Fact value for same Axis different AxisValues (PK).

Activation produces:
. Interaction : (Context, Interaction, Role, Actor / RoleState) : DCI
. Statement Resource URI encodes Interaction value (URI). Statement Interaction is Occurrence of parent Role Statement.
. Until Interaction definition (Contexts / use cases). Containment / Order (contexts interaction roles) hierarchies.

Instance, Fact, Interaction reified Containment Statements Occurrences points to the "parent" (containing) Dimensional Containment Statement.

Resource multiple Occurrences different Kinds.

Kinds type / state class model / Statements:

Statements : Occurrences of Resources (layers Subjects, Kinds)? Statement Occurrences role Kind: Statement ContextKind?

Kind type: Employee
. Type SK Statements: For all Kind Resource Occurrences Attributes / Values.
. SK (Employee) : (Ctx, SK :Person, Attrs, Values)*

Kind state: GoogleEmployee / HighSalaryEmployee / RemoteEmployee
. SK (GoogleEmployee) : (Ctx, SK :Employee, Attr :employer, Val :Google)
. Attr, Val: Placeholders / Kinds.

SK (Employee) : (Ctx, SK :Person, AttributeType, AttributeValue);
PK : (Ctx, PK :Parent, AttributeType, AttributeValue);
OK : (Ctx, OK :Parent, AttributeType, AttributeValue)

Ctx: Kind Occurrences (Kind as role)? PlayerType Resource?

Kind super / sub Kind (type / state) traversal.

Kind Resource Occurrence: Meta Kind.

Type Kind Statements: For all Kind Resource Occurrences Attributes / Attributes Values

Statement types:
Raw Statements (C, S, P, O).
Kind Statements (Ctk, Kind, AttributeType, AttributeValue).
Hierarchical Statements Instance: (Class, Instance, Attribute, Value).

Statements: Occurrences of Kind Resources.

Alignment (PK Player Predicates, SK Attributes?): (CK, S, P, O Materialization / Folding)
PK: (Predicate, SK, OK)? Folding?

Execution:
. Kind Functional & Kinds Application.
. Helpers: Index (FCA, Sets, etc.), Registry, Naming.
. Graph (Statement[]): Merge, Kinds aggregation, fold, unfold : Statement[]. By means of Kinds functional and Kinds application.
. Kinds aggregation, apply, merge, fold / unfold: Augmentation layers API.

Alignment:

Alignment:
. PK Player Predicates?
. SK Attributes?
. PK: (Predicate, SK, OK)?
. Infer: aggregate / deaggregate (fold / unfold).

Alignment Statements:

(Marriage, :aMarriage, :church, :aChurch)
(Marriage, :aMarriage, :date, :aDate)
(Marriage, :aMarriage, :husband, :aPerson)
(Marriage, :aMarriage, :wife, :anotherPerson)
(Churchs, :aChurch, :city, :aCity)
From / To Aggregation Statements?:
(Person, :aPerson :marriedWith :anoterPerson)
(Person, :aPerson, :marryDate :aDate)...

(Sales, :aSale, :store, :aStore)
(Sales, :aSale, :product, :aProduct)
(Sales, :aSale, :qty, :aQty)
(Sales, :aSale, :date, :aDate)
(Stores, :aStore, :region, :aRegion)
From / To Aggregation Statements?:
(Products, :aStore, :sold, :aProduct)
(Products, :aStore, :soldQty, :aQty)...

Activation Statements:

transferInteractionCtx: (MoneyTransfer, :aTransferCtx, :sourceAccount, :account123)
transferInteractionCtx: (MoneyTransfer, :aTransferCtx, :destinationAccount, :account456)
transferInteractionCtx: (MoneyTransfer, :aTransferCtx, :amount, :anAmount)
(AccountDebit, :transferInteractionCtx, :account, :account123)
(AcountDebit, :transferInteractionCtx, :amount, :anAmount)
(AccountCredit, :transferInteractionCtx, :account, :account456)
(AccountCredit, :transferInteractionCtx, :amount, :anAmount)
From / To: Alignment Statements?

Execution Pipeline Stream:
. Kind Functional & Kinds Application.
. Graph (Statements[]): Merge (Kinds Aggregation), Fold, Unfold : Statements[]. By means of Kind Functional and Kinds / Resources Application.

Kinds Aggregation, Apply, Merge. Fold / Unfold of (in Augmentation layers):
. Resources
. Occurrences
. Statements
. SPOs
. Kinds

Homoiconic (code as data) Approach

Resources (Occurrences, Statements, Contexts, Subjects, Predicates, Objects and their corresponding Kinds) are “functional” entities that can be “applied” to another Resources. The signature of such functional composition is as follows (TODO):

```
Resource::apply(res: Resource[]) : Resource[]
Resource::merge(res: Resource[]) : Resource[]
Resource::fold(res: Resource[]) : Resource[]
Resource::unfold(res: Resource[]) : Resource[]
```

Merging (Kind aggregated) input, unfolded and folded data against previously known data, inferring type, state and contextual relationships is what is meant with “code as data”.

Apply, Merge, Fold, Unfold in terms of Kind functional application. Kind::apply(ctx : Resource, attr : Resource) : Occurrence[]

Each Resource hierarchy class has their own overridden implementation of those ‘apply’ methods. The semantics of such “applications” (argument types and return types) are to be defined for each Resource type subclass implementing or overriding the corresponding ‘apply’ methods, their internal state and leveraging the following by means of Helper Services:

Apply as functional graph traversal by invocations compositions.

Kinds Functional Application.

FCA Inference / Traversal.

CPPE Contextual Inference.

Sets Model Representation Inferences.

Example: Kind applied to a Resource returns all Statements where the Resource is a player of the Kind.

Given a monadic wrapper (Spring Flux, Mono, for example) for handling streams processing:

```
aWrappedResource.map(aResourceInstance::apply) :
Flux<AResourceInstanceApplyType>
```

Then, Resource(s) applications can be chained and combined by means of stream pipeline composition.

Kinds Functional Comprehensions:

Kinds: extensional functions definitions. Functions Comprehensions: infer Kinds Functions from CPPE embeddings vía Helpers / ML. Resource Merge / Apply: Kinds inferred Functions inference. Kinds are functional given their occurrences in given dimensional contexts. Otherwise, an attribute could have multiple values.

Monadic merge / fold / unfold:

Augmentation pipeline wraps Resources in a stream invocation monadic chain where Resource / Augmentation (see below) merge, fold, unfold methods can be invoked in a functional manner.

Merge: application of aggregated (folded) / de aggregated (unfolded) Resources to output / input streams (augmented with previous merge results) on each key Context Kind (fold) aggregation completion and on each unfold stream scan emitted Resource.

TODO: Resource::apply Helpers Services approaches: FCA, CPPE, RDF inference, Sets, etc. ML Models / LLMs (classification : Aggregation, clustering : Alignment, regression : Activation). Unify Augmentation application Helper Services approaches APIs.

Materialization of Resources: State / IO

TODO: Refactor to IO / State Monad. Execution Resource Statements Schema.

“Primitive” Resources (Kinds) shall exist which allows for executable composition of streams models by their types and their internal instance state. Their signature may be a composite

traversal of application invocations. For example, for the “createLink” Resource primitive could be invoked like:

createLinkResource::apply(subjectRes : Resource)::apply(predicateRes : Resource)::apply(objectRes : Resource) : Statement Resource

Merge by means of Resource Application

Merge phase of pipeline agents processing first applies its input source / production Statement to each corresponding already known Statement. It then recursively applies resulting / inferred application result Resource(s) to the corresponding already known Statement structure composition.

Folding / Unfolding by means of Resource Application

Kinds layers (aggregation Statement types) folding and unfolding may be also defined in terms of Resource(s) functional application. Then the streaming pipeline can be built in terms of those operations (merge, fold and unfold) with streaming flows corresponding to each Augmentation pipeline step.

In this way, folding and unfolding would become “contextualized” operations, where each application’s context is the previously applied Resource.

Persistence and Endpoints Interactions by means of Resource Application

TODO: A schema shall exist which encodes Persistence IO actions and Endpoints Context Interactions behaviors and executions into Execution Statements Materialization / State IO performances.

TODO:
Resource hierarchy application semantics. Full layers types hierarchy (implement all layers models kinds). Template methods. Implementation Language? (Resources, Streams Processing), XSLT?

Streams Augmentation Services Agents Model

```
abstract class Augmentation
apply(input : Statement[]) : Statement[]
merge(input : Statement[]) : Statement[]
fold(input : Statement[]) : Statement[]
unfold(input : Statement[]) : Statement[]

class Aggregation extends Augmentation

class Alignment extends Augmentation

class Activation extends Augmentation
```

Each Pipeline layer (Aggregation, Alignment, Activation) has its own “template” methods.

Fold

Aggregate Dimensional Kinds Context Statements.
Stream “reduce” operation by means of aggregating by common roles.

TODO.

Unfold

De-Aggregate Dimensional Kinds Context Statements (Dimensional source Statements).
Streams “scan” operation emitting every unfolded Statement.

Merge

Identity Resolution driven. TODO: Find the way of unambiguously identifying concepts (Resources, Occurrences, Statements, SPOs, Kinds) equivalent (in a given role occurrence in a given context).

Merge: Reify as Resources Occurrences, Statements, SPOs, Kinds. Resolve ID of Resources, Contexts (ID, Type), Occurrences (IDs, Contexts, Roles).

Aggregated Processing / Production Statements acts as Templates / Upper ontology alignment schemas.

Layers context merged entities propagate upstream (unfolding) and downstream (folding).

Identity Encoding:

CPPE Embeddings
Index API.

FCA / Set Assertion Statements:
(Context, Concept, Role, Occurrence);
Concept, Context, Role, Occurrence recursively instance of Set Statements.

Naming Scheme:
Name assignment for inferred entities.
Registry Mapping of Merge encodings.

Messaging Infrastructure

Messaging pipeline:

Endpoints dataflow is configured in this way:

Datasources <-> Aggregation <-> Alignment <-> Activation <-> Dynamic Endpoints API

Datasources
(raw Statements in / raw Statements out)
<->
(raw Statements in / raw Statements out)
Aggregation
(Aggregation Statements out / Alignment Statements in)
<->
(Aggregation Statements in / Alignment Statements out)
Alignment
(Alignment Statements out / Activation Statements in)
<->
(Alignment Statements in / Activation Statements out)
Activation
(Activation Statements out / Activation Statements in)
<->
(Activation Statements out / Activation Statements in)
Dynamic Endpoints API

Messaging Format Representation: TreeGraph

All Base Model entities Statements of each layer type must encode all of their corresponding composing entities in a single message. A Statement containing a specific Kind must encode all its Kind's composing aggregation Statements, for example.

Source, Production Messages includes all Statements hierarchy which compose / lead to produced Statements (TreeGraph). Serialization. References to Resources (Registry Helper Service) to reduce message payload size..

Streaming Agents Augmentation Pipeline

The core of the framework is the Augmentation pipeline. It performs Aggregation, Alignment and Activation of input Resources in a streaming fashion.

Each pipeline Agent Service holds state given its previous interactions. Any of the three layers can be considered as a set of processed / known Resources (Statements) it knows about, given its layer corresponding level of abstraction.

Aggregation Service Agent

Context / Role / Type inference in the Aggregation Layer. Classification.

Aggregation Agent Domain Model

Data Layer. DOM (Dynamic Object Model)

Aggregation produces:
. (CK, S, P, O)
. Attribute / Instance? : (Class, Instance, Attribute, Value) : DOM

. Statement Resource URI encodes Instance value (URI). Statement Instance is Occurrence of parent Instance Statement.
. Until Attribute / Instance? definition (Classes). Instance Containment / Order (occurrence state / temporal? parts) hierarchies.

TODO.

Aggregation Agent Merge / Fold / Unfold

See: Homoiconic (code as data) approach.
TODO.

Alignment Service Agent

Ontology Matching / Links / Attributes / Dimensional / Order inference in the Alignment Layer. Clustering.

Alignment Agent Domain Model

Information Layer. N-ary Dimensional Relationships Modelling.

Alignment produces:
. (CK, SK, PK, OK)
. Axis / Fact? : (Dimension, Fact / MeasurementValue, Axis, AxisValue) : Dimensional Alignment
. Facts table with dimensions tables FKs. Graph / Quad Statements encoding.
. Statement Resource URI encodes Fact value (URI). Statement Fact is Occurrence of parent Fact Statement.
. Until Axis / Fact? definition (Dimension). Facts Containment / Order hierarchies.
. Ontology matching, links / attributes prediction: Same Fact value for same Axis different AxisValues (PK).

Alignment:

(Marriage, :aMarriage, :church, :aChurch)
(Marriage, :aMarriage, :date, :aDate)
(Marriage, :aMarriage, :husband, :aPerson)
(Marriage, :aMarriage, :wife, :anotherPerson)
(Churchs, :aChurch, :city, :aCity)
From / To:? (Person, :aPerson, :wife / :marriedWith, :anoterPerson)

(Sales, :aSale, :store, :aStore)
(Sales, :aSale, :product, :aProduct)
(Sales, :aSale, :qty, :aQty)
(Sales, :aSale, :date, :aDate)
(Stores, :aStore, :region, :aRegion)
From / To:? (Products, :aStore, :sold, :aProduct)

TODO.

Alignment Agent Merge / Fold / Unfold

See: Homoiconic (code as data) approach.
TODO.

Activation Service Agent

Contexts (Use Cases) and behaviors (Interactions) in the Activation Layer. Regression.

Activation Agent Domain Model

Knowledge Layer. DCI Contexts, Roles, Actors, Interactions.

Activation produces:
. Interaction / Role? : (Context, Role, Interaction, Actor / RoleState) : DCI
. Statement Resource URI encodes Role value (URI). Statement Role is Occurrence of parent Role Statement.
. Until Interaction / Role? definition (Contexts / use cases). Containment / Order (contexts interaction roles) hierarchies.

TODO.

Activation Agent Merge / Fold / Unfold

See: Homoiconic (code as data) approach.
TODO.

Helper Services

Monadic wrapper bound Base Model Entities (Resource, Occurrence, Statement, Context, Subject, Predicate, Object, Kind, ContextKind, SubjectKind, PredicateKind, ObjectKind) Functional APIs Definitions for stream processing.

“Leverage X” Sections Functional APIs.

Kinds as Extensional Function Definitions Functional APIs.

Resource merge / “application” (transform / template) Functional APIs.

Registry Service

Registry API Functional Definitions.

```
class RegistryService
   +  getAggregationGraph() : Graph
   +  getAlignmentGraph() : Graph
   +  getActivationGraph() : Graph

class Graph : Unified API view of Base Model Representations features
```

```
TODO
```

Naming Service

Naming API Functional Definitions.

Formal Grammars / LLM Models.
Names Relationships in Contexts / Identity Resolution (NER).
Representations Rendering / Parsing. Messages Representations.

TODO

Index Service

Index API FCA / CPPE / Sets Functional Definitions.

```
class IndexService
   +  getAggregationSubjectContext(subject : Resource) : Context
   +  getAggregationPredicateContext(predicate : Resource) : Context
   +  getAggregationObjectContext(object : Resource) : Context
   +  getAlignmentSubjectContext(subject : Resource) : Context
   +  getAlignmentPredicateContext(predicate : Resource) : Context
   +  getAlignmentObjectContext(object : Resource) : Context
   +  getActivationSubjectContext(subject : Resource) : Context
   +  getActivationPredicateContext(predicate : Resource) : Context
   +  getActivationObjectContext(object : Resource) : Context

class Context : Unified API view of FCA Contexts

TODO
```

State / IO Functional Addressing Features

State Monad.
IO Monad.

Datasources interactions.
Dynamic API Endpoints interactions.

Datasources

Raw RDF CSPO URIs Quad Statement Messages Endpoint. Configured to produce configured datasource data and to consume updated Statements synchronizing state.

Dynamic API Endpoints

Activation pipeline layer production messages I/O API Facade for rendering Dynamic API Endpoints. Execution Approach: State / Representational Resources Conversation IO.

Render an interface / protocol of discoverable Contexts behavior (Use Cases) and Interactions instances (executions) API of the unified integrated datasources facade by means of Activation activated knowledge message statements.

Develop this interface as a protocol for clients browsing available Contexts, instantiating Contexts Interactions into new Use Cases instances or browsing previous Use Cases Interactions in a conversational (browsing state) manner. Allow for binding produced results or navigation state back into Augmentation pipeline for further Augmentation, Contexts and Interactions inference.

State / IO Monad functional approach for a DCI Contexts Interactions generic API interface consuming and producing Activation pipeline layer production messages. Messages fan in / fan out Augmentation pipeline for further execution and inference (merge, unfold, fold).

Highlight how Activation pipeline layer production messages are rendered as API schema, instances and inferences and how user interactions with this rendered interface modifies or produces new knowledge statements accordingly and how the Dynamic Endpoints communicate with Activation pipeline layer via a State / IO Monad approach for input / output / materialization of Activation production messages.

—

The "Dynamic API Endpoints" section of the document must outline how a generic API Facade could expose Activation layer Contexts, Roles, Interactions and Actors workflows (Use Cases, Use Case Instances) that can be browsed and executed by the API Resources rendered representations navigation means.

API Facade endpoints produced and consumed API Resources rendered representations (REST) conveys all the navigational “state” of a “session” (State Monad). These representations are built from and rendered to the Activation layer Context production Statements: (DCIContextCK, RoleSK, ActorPK, InteractionOK). Leverage an State Monad pattern for navigational context and Augmentation pipeline state and an IO Monad for Activation layer Context production Statements transforms into API Endpoints rendered resources representations.

The API endpoints receive and produce API Representations rendered resources. They receive an API Representation rendered resource, transforms it into Activation layer Context production Statements, performs Activation (fans in / fans out all the Augmentation pipeline layers for inference) and renders back the updated Activation layer knowledge into API Representations rendered resources format.

This will be the “state” flowing across the rendered API endpoints resource representations and the backend Activation Layer message format in which API endpoints interactions

resource representations are transformed back and forth (IO Monad). The initiating message (Resource) from a session could be the selection of a top level Context (Use Case). The client retrieves, inspects and posts back this representation and the response could be a list of the Use Case instances (Interactions) and a “New Interaction…” placeholder Resource. Then follow the same pattern for the other resource types: post a representation and obtain a list of available resources (parent, child, placeholder) for this resource type.

The "Generic Dynamic API Endpoints Browser / Designer" must outline

POST /contexts
POST /contexts/{context}
POST /contexts/{context}/interactions
POST /contexts/{context}/interactions/{interaction}
POST /resources/{resource}

—

The Dynamic API Endpoint layer is the protocol boundary between the Activation layer and external clients. Activation produces Context/Role/Actor/Interaction Statements of the form `(DCIContextCK, RoleSK, ActorPK, InteractionOK)`. The Dynamic API consumes these activated Statements to render discoverable Context and Interaction representations and produces Interaction requests and navigation/binding events that are submitted back to Activation.

A State/IO Monad provides the functional execution model for this boundary. **State** represents the current API/interaction navigation state, including selected Contexts, Context instances, Actors, Roles, Interactions, execution results, bindings and navigation history. **IO** represents communication with the Activation message stream and its asynchronous production/consumption of Activation Statements.

Conceptually:

InteractionRequest
+
ActivationApiState
↓
State / IO computation
↓
Activation input message
↓
Activation processing
↓
Activation production message(s)
↓
updated ActivationApiState
+
InteractionResult

The Dynamic API renderer projects the resulting activated Statements and State into a discoverable HAL-like representation. API operations do not constitute a separate domain model; they are projections and interactions over Activation knowledge.

Dynamic API Endpoints define the **runtime protocol** by which activated Contexts, Roles, Actors, Resources and Interactions are discovered, instantiated, executed and observed.

The State/IO Monad applies to the Activation ↔ Dynamic API boundary; persistence and synchronization with underlying datasources remain concerns of the augmentation/data-source integration architecture and are not the responsibility of this API monad.

Dynamic API Endpoints Integration

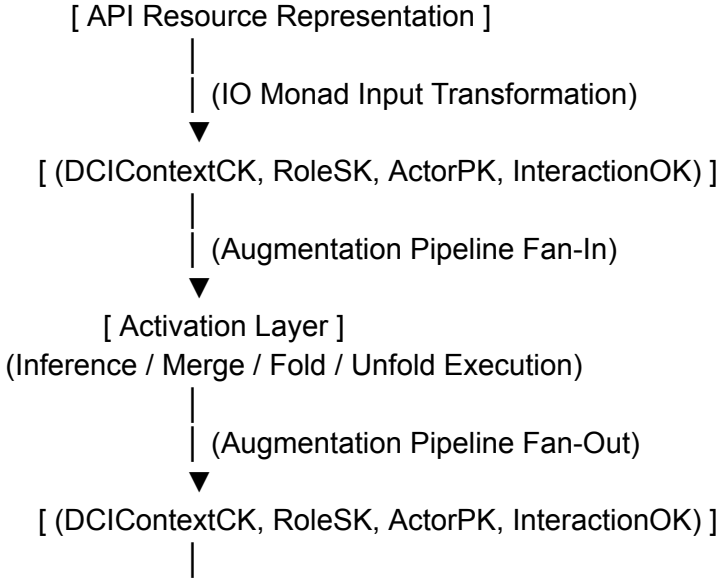
The Dynamic API Endpoints section defines the runtime execution boundary between external consumers and the underlying semantic stream processing infrastructure. Implemented as an I/O API Facade, this layer projects activated knowledge statements into discoverable, executable interface representations while capturing client interactions and state transitions to feed back into the reactive augmentation stream.

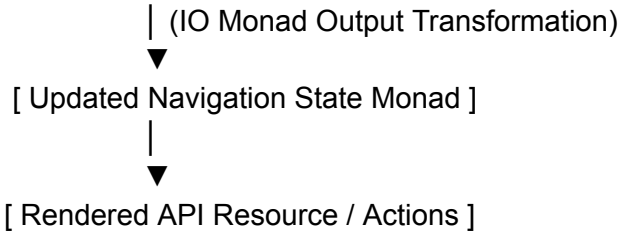
Monadic Context & Interaction Pipeline

The boundary between the Activation layer and the Dynamic API is governed by a combined **State / IO Monad** operational model:

- **State Monad**: Manages the runtime navigational state of a client session. It encapsulates active session variables, selected top-level Contexts (Use Cases), Interaction instances, Actor assignments, active Roles, execution parameters, and historical navigation paths across resource representations.
- **IO Monad**: Handles the asynchronous, message-driven transformations between rendered REST/HAL resource representations and the backend Activation layer production Statements:

$$ActivationStatement = (DCIContextCK, RoleSK, ActorPK, InteractionOK)$$





Protocol Execution & Resource Traversal

The API Facade operates through a continuous RESTful resource navigation loop. Navigational state is conveyed entirely within the rendered resource representations, allowing clients to browse, inspect, and execute domain behaviors dynamically:

1. **Top-Level Discovery**: The client initiates a session by requesting the top-level Context root. The API responds with a representation listing available top-level Use Cases ($DCIContextCK$) and discovery hypermedia links.
2. **Context Selection & Instance Enumeration**: Inspecting and posting a selected Use Case Context statement transforms the representation via the IO Monad into an Activation layer message. The Activation layer processes the message across the augmentation pipeline (fanning in/out through Aggregation, Alignment, and Activation) and returns updated knowledge statements.
3. **Resource & Interaction Navigation**: The response renders a representation containing:
    - A list of active Use Case instances ($Interactions$).
    - Available participant Roles ($RoleSK$) and bound Actors ($ActorPK$).
    - A "New Interaction..." placeholder Resource representing potential execution boundaries.
4. **State Transition & Action Execution**: Submitting a payload to a placeholder or interaction resource triggers backend execution. The payload is converted into Activation production Statements, evaluated for state transitions (leveraging Previous, Current, Next networks and CPPE state transition products), and persisted across synchronized datasources. The updated state is rendered back to the client as new hypermedia controls and resource representations.

Generic Dynamic API Endpoints Browser / Designer

The dynamic endpoints facade exposes two primary operational modes for generic client interaction:

- **Generic Semantic Browser**: A discoverable, hypermedia-driven UI layer that allows users to traverse available Use Cases, inspect active Context instances, view bound Roles/Actors, and trigger executable Interactions interactively.
- **Context / Alignment Designer**: An interactive interface enabling administrators and domain designers to bind newly discovered graph resources, interaction results, and navigation states back into the reactive Augmentation Pipeline. These bindings generate new aligned rules, refine DCI Context definitions, and establish updated execution templates for continuous semantic augmentation.

Dynamic API Endpoints API Architecture & Monadic Workflow

The generic **API Facade** exposes the Activation layer's Contexts, Roles, Interactions, and Actors workflows (representing Use Cases and Use Case Instances) through discoverable REST resource representations. Client applications navigate, inspect, and execute these workflows directly via hypermedia navigation controls embedded within the rendered resource representations.

Monadic Execution Model: State and IO Monads

The API Facade operates on a dual Monadic functional foundation:

- **State Monad**: Encapsulates all navigational state and session context across API resource interactions, tracking current Context selections, active Roles, bound Actors, execution histories, and Augmentation pipeline state.
- **IO Monad**: Handles the asynchronous side-effects, message transformations, and I/O streams between the REST endpoints and the backend Activation Layer message pipeline.

API Resource Representation (REST)
  │ (IO Monad: Unfold / Transform)
  ▼
Activation Production Statements: (DCIContextCK, RoleSK, ActorPK, InteractionOK)
  │ (Fan-In / Fan-Out: Augmentation Pipeline Inference)
  ▼
Updated Knowledge & Inferred State
  │ (IO Monad: Fold / Render)
  ▼
Updated API Resource Representation + Navigation Means (State Monad)

Activation Statements Transformation Flow

The REST representations consumed and produced by API endpoints are anchored to the Activation layer production Quad Statements:

$$ActivationStatement = (DCIContextCK, RoleSK, ActorPK, InteractionOK)$$

The transformation and execution pipeline across the API endpoints proceeds as follows:

1. **Receive Resource Representation**: The endpoint receives an incoming API Representation resource conveying the client's current session state, submitted parameters, or selected navigational action.
2. **Transform to Activation Statements (IO Monad)**: The IO Monad transforms the REST resource representation into Activation layer Context production Statements $(DCIContextCK, RoleSK, ActorPK, InteractionOK)$.
3. **Activation & Augmentation Pipeline Processing**: The statement is submitted into the Activation layer, triggering a fan-in/fan-out across all Augmentation pipeline layers (Aggregation, Alignment, Activation) to perform inference, type resolution, link completion, and state transition calculations.

4. **Render Updated Knowledge (IO Monad)**: The updated knowledge statements and inferred states produced by the pipeline are rendered back into API Resource rendered representations.
5. **Navigational State Projection (State Monad)**: The State Monad threads updated hypermedia navigation controls, parent/child relationships, available actions, and placeholder resources back to the client representation.

Conversational Session & Resource Navigation Pattern

The API Facade enforces a uniform, stateful REST browsing pattern across all resource types (Context, Role, Actor, Interaction):

1. **Top-Level Context Selection (Use Case Discovery)**:
    - **Initiating Request**: A session begins when the client selects a top-level Context resource representing a Use Case (e.g., :CustomerOnboardingDCIContextCK).
    - **Client Inspection**: The client retrieves and inspects this representation, which embeds available Roles, bound Actors, and state transition choices.
2. **Context Instances & Interactions Listing**:
    - **Post Representation**: The client posts back the Context representation to instantiate or navigate the selected Use Case.
    - **Facaded Response**: The API responds with a list of active Use Case instances (Interactions) along with a canonical **"New Interaction..."** placeholder Resource.
3. **Recursive Sub-Resource Traversal**:
    - Following the same pattern for all other resource types (Role, Actor, Interaction), posting a resource representation yields a list of available related resources:
        - **Parent Resources**: Enclosing Contexts and parent Roles.
        - **Child Resources**: Active sub-interactions, bound Actors, and child state transitions.
        - **Placeholder Resources**: Uninstantiated template representations (e.g., "New Role...", "New Actor...", "New Interaction...") enabling dynamic binding and creation.

By projecting Activation Statements into REST hypermedia, the API Facade maintains bidirectional synchronization between UI interactions and backend pipeline knowledge graphs.

Dynamic API Endpoints Summary

The Dynamic API Endpoint layer is the protocol boundary between the Activation layer and external clients. Activation production messages provide the executable knowledge from which the API Facade dynamically renders discoverable resources and interaction affordances. The fundamental Activation Context production Statement is:

(DCIContextCK, RoleSK, ActorPK, InteractionOK)

where the Context represents a Use Case, Roles represent the participating behavioral roles, Actors represent the participating execution resources, and Interactions represent executable Use Case behavior. The existing facade objective is therefore extended from merely exposing activated Statements to exposing their **rendered Resource representations as the navigational state of a REST session**.

A generic API Facade does not introduce another domain model. It renders Activation Resources, Contexts, Roles, Actors and Interactions as API Resources and exposes the relationships between those resources as navigational links and executable affordances. A client discovers a Use Case by following representations rather than by knowing a fixed endpoint vocabulary in advance. This preserves the HAL-like, discoverable character already established for the Unified Systems Facade.

Resource Representation and Navigational State

An API Resource representation is the externally visible projection of an Activation-layer Resource and its current navigational context. The representation contains both the resource data and the links or affordances through which the client may continue browsing or execute an Interaction.

Conceptually:

Activation Statements
↓
Resource / Occurrence / Statement materialization
↓
API Resource representation
↓
client inspects representation
↓
client follows a navigational affordance
↓
next API Resource representation

The representation therefore carries the **state of the session**. The URL alone is not the complete state of the interaction; the current representation, its selected Context, resource identity, available child resources, execution bindings, results and navigation affordances together constitute the navigational state.

A representation may consequently contain relations such as:

self
parent
context
roles
actors
interactions
instances
new

result
next
previous

The concrete URI layout is implementation-defined. The semantic identity of the resources and the transition between them is determined by the Activation Statements and by the current API State.

State Monad: API Navigation and Augmentation State

The API Facade uses a **State Monad** to model the navigational state flowing across successive rendered API representations.

Conceptually:

```
State =
{
   selected Context,
   selected Role,
   selected Actor,
   selected Interaction,
   Context instance,
   Interaction instance,
   bound Resources,
   execution Result,
   navigation history,
   Augmentation bindings
}
```

A navigation operation transforms one API State into another:

```
navigate(State, Representation)
      → State'
```

The resulting State is then rendered into the next API Resource representation. Consequently, a representation returned by one endpoint can become the input representation to the next endpoint without requiring the client to reconstruct hidden session information.

This makes the API Resource representation itself the transferable **navigation state** of the session. The client retrieves a representation, inspects its available Resources and affordances, selects or modifies a representation, and posts that representation to request the corresponding next state.

The same State Monad also carries the state accumulated by the Augmentation pipeline interaction: selected Contexts, inferred Roles, bound Actors, discovered Interactions, produced results and any new statements that become available through execution.

IO Monad: API Representation ↔ Activation Statements

The **IO Monad** models the effectful boundary between the rendered REST resources and the Activation message stream.

The transformation is bidirectional:

API Representation
↓
Activation Context Production Statements
↓
Activation
↓
fan-in / fan-out Augmentation Pipeline
↓
new / updated Activation Statements
↓
API Representation

The API therefore does not directly execute business logic independently of the Augmentation Pipeline. An incoming API Resource representation is transformed into Activation-layer Context production Statements. Activation then merges, unfolds, folds and publishes the resulting knowledge through the same message-driven pipeline used by the other augmentation layers. The resulting Activation production messages are materialized back into API Resources and rendered as the response representation. The pipeline is explicitly defined as an asynchronous fan-in/fan-out stream in which production Statements may become inputs to subsequent processing and inference.

Conceptually:

InteractionRequest
+
ActivationApiState
↓
State / IO computation
↓
API Representation → Activation Statements
↓
Activation processing
↓
merge / unfold / fold
↓
Activation production Statement(s)
↓
Activation Statements → API Representation
↓
updated ActivationApiState
+
InteractionResult

Thus:

State = navigational and augmentation session state

IO = effectful transformation and communication
between API Resources and Activation Statements

The State Monad determines **where the client is and what is currently known in the session**. The IO Monad determines **how that state is materialized into and out of the Activation message stream**.

Generic Contexts / Roles / Actors / Interactions Resources

The generic facade exposes the Activation hierarchy as navigable API Resources rather than as fixed application-specific controller methods.

A top-level Context Resource represents an inferred Use Case:

Context
↓
Roles
↓
Actors
↓
Interactions
↓
Interaction instances

A Context representation may therefore expose:

/context/{context-id}

```
{
   "id": "...",
   "kind": "DCIContextCK",
   "name": "...",
   "state": "...",
   "_links": {
      "self": "...",
      "parent": "...",
      "roles": "...",
      "actors": "...",
      "interactions": "...",
      "new": "..."
   }
}
```

The exact representation format is not normative; the important property is that the rendered Resource contains sufficient state and navigation information to continue the interaction.

A Role Resource identifies the Role participating in the selected Context:

/context/{context-id}/roles/{role-id}

An Actor Resource identifies the Actor associated with that Role and Context:

/context/{context-id}/actors/{actor-id}

An Interaction Resource identifies an executable behavior:

/context/{context-id}/interactions/{interaction-id}

An Interaction instance represents a concrete execution of a Use Case:

/context/{context-id}/interactions/{interaction-id}/instances/{instance-id}

The new affordance is itself a Resource representation rather than a special out-of-band command. Following or posting to this Resource requests materialization of a new Context, Role, Actor or Interaction instance.

Browsing a Use Case as REST Resource Navigation

The initiating Resource for a session may be the selection of a top-level Context, which corresponds to a Use Case.

For example:

```
GET /contexts
      ↓
list of Context Resources
      +
"New Context..." Resource
```

The client retrieves a Context representation:

GET /contexts/{context}

and receives a representation containing the Context state and the available navigation affordances:

```
Context Resource
   parent
   child Roles
   child Actors
   child Interactions
   previous instances
   "New Interaction..." placeholder
```

The client then posts the inspected representation, or the representation of the selected Context state:

POST /contexts/{context}

↓
Activation Context production Statements
↓
Activation
↓
Interaction Resources

The resulting representation may contain:

```
{
   "context": "...",
   "instances": [
      "...InteractionInstance1...",
      "...InteractionInstance2..."
   ],
   "_links": {
      "parent": "...",
      "context": "...",
      "new": ".../interactions/new"
   }
}
```

The important property is that the response is **not merely a collection returned by a query**. It is the next navigational state of the session. It contains the available parent, child, previous-instance and placeholder Resources from which the client can continue.

The same protocol applies recursively to all resource types:

POST representation
↓
Activation transformation / execution
↓
list of available Resources
↓
(parent)
(child)
(previous instance)
(new placeholder)
(next affordance)

Consequently, a generic client need not contain prior knowledge of the individual application's Context, Role, Actor or Interaction endpoint structure. It follows the Resources rendered by the current representation.

Resource Rendering from Activation Production Statements

The Activation production Statement:

(DCIContextCK, RoleSK, ActorPK, InteractionOK)

is the source structure from which the facade renders Context and Interaction Resources.

The rendering process may be understood as:

```
(DCIContextCK, RoleSK, ActorPK, InteractionOK)
                    ↓
              Resource materialization
                    ↓
          Context Resource representation
                    ↓
      Roles / Actors / Interactions links
                    ↓
       executable Interaction affordances
```

Conversely, when a client posts an Interaction representation:

```
API Interaction Resource
      ↓
Resource → Occurrence → Statement transformation
      ↓
(DCIContextCK, RoleSK, ActorPK, InteractionOK)
      ↓
Activation
```

the posted representation becomes an Activation message rather than being treated as an unrelated REST command.

This preserves the document's homoiconic principle: Resources, Statements and Kinds can participate in functional application and composition, while fold/unfold and merge operations provide the means for carrying those transformations through the streaming pipeline.

Executing an Interaction

An Interaction Resource is both descriptive and executable. Its representation identifies the currently applicable Context, Role and Actor state and exposes an execution affordance.

Conceptually:

```
GET /contexts/{context}/interactions/{interaction}
      ↓
Interaction Resource representation
      ↓
POST representation
      ↓
Interaction execution request
      ↓
Activation input message
      ↓
Activation
```

↓
updated Context / Interaction state
↓
result representation

Execution may cause the Activation layer to unfold the selected production Statement into the underlying Rule and Property Statements, infer or complete additional state, and fan the resulting messages back through the pipeline. This is consistent with the existing execution model in which Activation production Statements can be consumed again as inputs and unfolded into their originating structures.

The response therefore renders the **updated Activation knowledge**, not merely an acknowledgment that the HTTP request was accepted.

For example:

POST /contexts/customer-onboarding/interactions/initiate

request:
Interaction Resource representation

response:
Interaction Instance Resource representation

```
{
   "state": "...",
   "result": "...",
   "_links": {
      "self": "...",
      "context": "...",
      "parent": "...",
      "next": "...",
      "previous": "..."
   }
}
```

An executed Interaction becomes a navigable Resource that can be inspected, revisited and used as the starting representation for the next interaction.

Parent, Child and Placeholder Resources

The generic facade treats browsing as a uniform operation over Resource representations.

For each current Resource representation, the facade may render:

Current Resource
├── parent Resource
├── current Resource
├── child Resources

```
    ├── previous instance Resources
    └── "New ..." placeholder Resource
```

The placeholder is significant because it represents an **available state transition** rather than a missing Resource. A `New Interaction...` Resource means that the current Context supports materialization of another Interaction instance.

The same pattern applies to:

Contexts
Roles
Actors
Interactions
Interaction Instances
Resources bound to an Interaction
Execution Results

The generic API browser can therefore be implemented without specialized screens for each discovered Use Case. It renders the Resources and affordances contained in the current representation and follows the state transitions they describe.

API Facade and Augmentation Pipeline Feedback

API execution is also an input to the inference architecture. A representation posted by the client can contain selected Resources, bindings and execution results that become new Activation inputs. These messages can fan back into the Augmentation Pipeline so that execution state contributes to subsequent merge, fold and unfold operations and can lead to further inferred Contexts, Roles and Interactions.

API session
↓
rendered Resource representation
↓
Activation production Statement
↓
Activation
↓
Augmentation fan-out
↓
new inferred knowledge
↓
Activation production Statement
↓
API Resource representation

The facade consequently forms a bidirectional boundary rather than a terminal presentation layer:

Datasource statements

↓
Aggregation
↓
Alignment
↓
Activation
↓
Dynamic API
↓
client navigation / execution
↓
Activation
↓
Alignment / Aggregation / Activation
↓
updated knowledge
↓
Dynamic API

This retains the framework's principle that the unified facade is generated from inferred semantic Resources, Kinds, Relationships, Contexts, Roles, Actors and Interactions, while Resource views expose integrated state and Interaction endpoints expose executable Contexts and state transitions.

Generic Dynamic API Protocol

The resulting protocol can therefore be summarized as:

1. GET a representation
2. Inspect its state and navigation affordances
3. Follow a parent, child, instance or "new" Resource
4. POST the selected Resource representation
5. Transform representation → Activation Statements
6. Activate and fan-in / fan-out the Augmentation Pipeline
7. Materialize resulting Activation Statements
8. Render the updated Resource representation
9. Continue navigation from the returned representation

The API endpoint set is therefore **generic and dynamically generated**. Endpoint behavior derives from the Resources and Activation Context Statements currently known by the system rather than from a statically authored controller hierarchy.

Generic Dynamic API Endpoints Browser / Designer

Render a default Dynamic API Endpoints User Interface allowing for Use Case Contexts, Behavior Interactions and Augmentation pipeline alignments and bindings browsing of available knowledge.

Allow for binding produced results or navigation state back into Augmentation pipeline for further Augmentation, Contexts and Interactions inference.

Generic Dynamic API Endpoints Browser / Designer defines the **reference client** for that protocol: a generic semantic browser for discovering and executing activated capabilities, plus a designer for binding resources, interaction results, navigation state and augmentation outputs into new executable Contexts and Interactions.

The **Generic Dynamic API Endpoints Browser / Designer** is the reference client for this protocol. It renders the current API Resource representation, exposes its navigation affordances, permits Context / Role / Actor / Interaction browsing and execution, and allows produced results, resource bindings and navigation state to be submitted back into the Augmentation Pipeline.

It may consequently act simultaneously as:

Semantic Resource Browser
+
Use Case / Context Browser
+
Interaction Executor
+
Interaction Instance History
+
Augmentation Binding Designer

The browser/designer is not a separate semantic authority. It is a generic client over the Dynamic API protocol; the authoritative semantic state remains the Activation-layer Resources and Statements from which the representations are rendered.

Implementation Techniques

The framework's primary objective is to build a semantic, RDF-based data integration and intelligence system. It processes raw CSPO (Context, Subject, Predicate, Object) RDF graph streams through a reactive three-stage Augmentation Pipeline—**Aggregation** (data/type/role inference), **Alignment** (information/ontology matching, links, ordering), and **Activation** (knowledge/contexts, DCI behaviors, interactions)—to yield executable domain models, exposing a unified Contexts and Interactions API while maintaining real-time synchronization with underlying source datasources.

The techniques below are complementary implementation mechanisms for the same Base Model and Augmentation Pipeline. Each technique provides a computational lens for one or more operations—representation, traversal, classification, matching, inference, execution, identity, or interaction—while results are ultimately expressed as Resources, Occurrences, Statements and Kinds and propagated through Aggregation, Alignment and Activation.

Leverage TMRM (ISO Topic Maps Reference Model)

Underlying graph metamodel.

- **Metamodel Foundation**: Serves as the underlying, subject-centric graph metamodel for representing Resource, Occurrence, Statement, and Kind base structures.
- **Contribution to Framework Objective**: Provides an implementation-independent semantic substrate that handles resource identity, graph traversal, reified statement management, and context-preserving access. This allows the Aggregation, Alignment, and Activation pipeline layers to share a unified graph representation without losing provenance or context.

Use the ISO Topic Maps Reference Model (TMRM) as the underlying graph metamodel for the Base Model, providing a subject-centric and implementation-independent foundation for representing `Resource`, `Occurrence`, `Statement`, `Subject`, `Predicate`, `Object`, and their associated `Kind` structures. The Base Model can be mapped onto TMRM subject proxies and properties, with Statements providing the reified graph relationships and Occurrences providing the contextual roles in which Resources participate.

TMRM can therefore provide the semantic graph substrate and generic graph operations required by the service, while the Base Model and Augmentation Pipeline define the application-specific semantics and processing layers. In particular, the metamodel should support APIs for:

- **Resource / subject management:** create, identify, retrieve, update and merge graph resources/subject proxies, including stable URI-based identity.
- **Statement management:** create, retrieve, remove and traverse reified `S-P-O` Statements and their contextual Occurrences and roles.
- **Graph traversal:** navigate subjects, predicates, objects, occurrences, contexts and their relationships, including recursive traversal of Kinds and Statements.
- **Context-preserving access:** retrieve the statements and properties associated with a subject while preserving the context in which each relationship or occurrence was asserted.
- **Property and kind access:** expose the properties of a resource and the Statements defining `SubjectKind`, `PredicateKind` and `ObjectKind`, supporting the classification structures used by the Aggregation phase.
- **Pattern/query access:** match graph structures and Statement patterns so that Augmentation Pipeline agents can discover applicable processing Statements and apply them to input Statement streams.
- **Identity and merging:** provide subject identity/merging facilities that allow information about the same underlying subject from different sources or streams to be consolidated without losing its provenance/context.
- **Schema/constraint support:** expose the metamodel structures needed by higher-level services to validate or constrain the graph, while leaving the specific application constraints to the service model.

The TMRM layer is thus primarily responsible for **representing and accessing the semantic graph**, rather than implementing the Aggregation, Alignment or Activation

semantics themselves. Those services consume and produce the graph structures through the metamodel APIs, allowing the same underlying graph representation to support the DOM, Dimensional and DCI layers described by the Augmentation Pipeline.

This also provides a natural foundation for the existing FCA and CPPE components: FCA can operate over the subject/property structures exposed by the graph, while CPPE can use Resource/Statement identities and identifiers as inputs to its numerical inference mechanisms.

TMRM supplies the semantic substrate; the augmentation layers supply the application-specific inference semantics.

Leverage Dimensional Features

See: “Sets layout for Dimensional Analysis”
See: “Context’s ContextKind(s) Dimensional Roles”

- **Dimensional & State Modeling**: Utilizes power set attribute inclusions to define ordering and hierarchies, alongside PCN (Previous, Current, Next) state transition networks for graph node flows.
- **Contribution to Framework Objective**: Directly powers the **Alignment** (axis ordering and dimensional relationships) and **Activation** (state transition networks and behavioral flows) pipeline stages, enabling raw data mutations to be interpreted as structured, executable business interactions.

Purpose

Represent semantic differences as axes, dimensions, states and transitions.

Core mechanisms

- Attribute power sets.
- Subset/superset ordering.
- Kind hierarchies.
- Dimensional axes.
- Previous/Current/Next (PCN) transitions.
- Reified relationships.

End-to-end contribution

Aggregation: infer Kinds and type/state hierarchies from recurring attributes.
Alignment: compare entities across dimensions and establish ordering/analogy.
Activation: represent state transitions and contextual behaviors.

Dimensional features turn static graph structure into an ordered semantic space in which equivalence, extension, transition and analogy can be represented explicitly.

Statement Predicates are the Type or Instance of an Action / Verb

:name instanceOf :naming

The Power Set of every attribute contains the sets of attributes of every possible class / concept. Includes relation defines an ordering over the power set. Power Sets (CSPO / Reified Types Sets). All possible Subjects, Predicate, Objects, Kinds Grouped Types. Order Relationship (Includes). CSPO Sets Intersection: All possible Statements. Grammar (production from possible rules).

Order and hierarchies are defined by subset / superset relationships. An object having attributes who are a superset of another object attributes is “extending” or “after” this object. Dimensional (Contexts) Features

Kinds, type / state hierarchy / order inference and materialization into Statements.

Alignment. Order (axis arrangements). State transitions flow. Infer and encode types / instances state transition networks on a given context on a given event:

- PCN (Previous, Current, Next) in axis / scope Graph Traversal Node Types: Individuals, States (occurrences role types), Associations (events). Order Hierarchies. Contexts:
    - (Single, Marriage, Married);
    - (Married, Divorce, Divorced);
    - (Single, Married, Divorced);
    - (Junior, Promotion, Semisenior);
    - (Junior, Semisenior, Senior);
    - (Unemployed, Employment, Employee);
    - (Unemployed, Employed, Unemployed);
    - (John, successor, Peter);

Leveraging Grammars as an LLM Interaction tool

Possible contexts / functional transitions (behavior) as formal grammars rules / productions. Possible Prompts in context. Contexts Roles behaviors Flows. Inference context productions from grammars. Statements as grammar rules / productions.
Infer Occurrence Type.
Retrieve Occurrence Types.
Retrieve Type Occurrences
Infer Statement Dimensions.
Retrieve Statement Dimensions.
Retrieve Dimension Statements.

Folding and unfolding Merge operations can leverage comparisons of Resources for sorting and matching. This is sorting in a nested contexts approach where one sorted occurrence may be the ‘parent’ of child sorted occurrences (happened and sorted in the parent occurrence scope). Resource application can be used for sorted structures traversal and further comparisons.

TODO: For leveraging CPPE, Algebraic Embeddings and FCA the comparison results may be encoded in a 3 bit string mask (octal) representation, for example: 001 is less than, 100 is greater than, 111 is equal, etc.

One goal would be to be able to encode comparison results as Merge Materialized Merge Traversal Comparison Statements Results. Example: A to B in Context C is like D to E in Context F.

The means for being able to construct such Statements for comparisons is by means of PredicateKind. Reified Relationships aggregation:

(:subject, :employer / :position / :salary, :objects...)

-> (:subject, :employment, :anEmployment)

-> (:anEmployment, :attributes..., :values…)

:employment: WorkRelationship PredicateKind (materialize)
:anEmployment: Employment Relationship Resource instance Kinds (ObjectKind, SubjectKind).

Attributes: Dimensional Axes. Values: PredicateKind ValueType Aggregation. Infer and materialize :anEmployment reified relationship object / subject for :employment predicate of WorkRelationship PredicateKind.

PredicateKind of SubjectKind Kind Attribute (Employee, Student) / ObjectKind Kind Value (Employer, School).

TODO: Infer
Employee is to Employer (in WorkRelationship) as Student is to School (in StudentshipRelationship).

Materialize: (Resource, ComparisonEncoding(axis, binary mask), Resource).

Merge (materialize) Resources according to their axes binary mask comparison results.

Leverage CPPE, Algebraic Embeddings, FCA.

Augmentation Layers Kinds Relationships Reification

Augmentation Layers, particularly the Information or Alignment layer already performs such a PredicateKind relationship, aggregating those Kinds from Subject and Object Kinds. Knowledge Activation Layers performs the aggregation over the previously aggregated layer results. Such arrangements should allow for: Information Alignment axes ordering and matching and Knowledge Activation axes ordering and matching.

The examples shown so far may be appropriate for Information Alignment Layer instance data comparisons and inference. The Knowledge Activation layer should materialize ordering and matching of reified contexts and interactions of behavior instance data.

Leverage Formal Grammars

Grammatical Production Rules: Maps Non-Terminals to Kinds, Terminals to Resources, and Production Rules to pipeline Statements using monadic parsers.

Contribution to Framework Objective: Formalizes stream processing as a dynamic grammar expansion process. Raw statements are parsed into functional non-terminal concepts (**Aggregation**), matched and ordered through production rules (**Alignment**), and evaluated to generate prompts, rules, and interaction executions (**Activation**).

End-to-end contribution

Aggregation: infer grammar rules from observed Statements.
Alignment: determine valid/analogous productions and their dimensional ordering.
Activation: start from a contextual Statement and execute applicable productions.

Layered Production Rules Composition:

Formal Grammars Rules Productions for Classification, Matching, Completion, Order / Comparisons and potential Contexts / Interactions Rules Productions.

Non Terminals: Kinds
Terminals: Resources
Production Rules: Aggregation, Alignment, Activation layers input / source, output / production Statements.
Start Symbol: (Occurrence, Occurrence, Occurrence, Occurrence):
TODO: Refactor class model. Context, Subject, Predicate, Object are the roles in the Statement determined by the Kind of the Occurrence.

Rules Productions: Layered Rules Production Statements materialization. Contextualized, until no non-terminals: Non Terminals: Functional Relationships (functional merge).

Monadic parser. Dynamic grammars / DSLs (upper Grammars / Contexts / Kinds aligned). Rules / Productions : Schema / Occurrences.

Aggregation:
Infer Rules (Grammar) from Productions (Statements).

Alignment:
All possible Grammar Production Rules Statements, all actual / valid grammar rules production Statements (matches Functional Relationships / Kinds merge).
Order / Dimensions: Rules Productions as part / step of another Production Rule.

Activation:

Prompt: Statement (from contextual possible start statements interactions list), Merge: Abstract Statement Production Rules (Aggregation), Unfold: scan all possible Statements from abstracted Statement Production Rules (Alignment). Perform Productions from Rules starting at scanned possible Statements.

Leverage RDF, RDFS, OWL and SPARQL

Standardized Graph Infrastructure: Employs raw RDF Quad/Triple stores, SPARQL endpoints, and standard W3C semantic web ontologies.

Contribution to Framework Objective: Guarantees universal interoperability and ingestion capabilities from any legacy database or backend capable of exporting RDF graph representations, enabling seamless graph pattern matching across heterogeneous data streams.

Purpose

Exploit existing semantic-web infrastructure for:

- graph querying,
- schema/type representation,
- inference,
- constraints,
- external interoperability.

End-to-end contribution

Aggregation: query source statements and structural patterns; derive or validate types.
Alignment: use graph patterns and ontology constructs to find correspondences and relationships.
Activation: query discovered contexts, roles and interactions and persist resulting statements.

RDF is the representation; RDF / RDFS / OWL / SPARQL are implementation capabilities operating on that representation.

Leverage Sets Model

See: Sets layout for Dimensional Analysis

Mathematical Set Primitives: Defines Contexts, Subjects, Predicates, Objects, and Kinds as mathematical sets, applying union, intersection, and relative difference operations.

Contribution to Framework Objective: Delivers an ultra-fast, deterministic, low-level merge primitive for stream processing, facilitating immediate deduplication and exact-match aggregation before higher-level lattices or embeddings are computed.

Purpose

Provide deterministic algebra over:

- Subjects,
- Predicates,
- Objects,
- Contexts,
- Kinds,
- Statement components.

Core operations

- Union
- Intersection
- Difference
- Inclusion / subset
- Compatibility

End-to-end contribution

Aggregation: intersection/commonality → type inference.
Alignment: set inclusion and difference → structural correspondence.
Activation: sets of possible states/transitions → candidate execution structures.
Merge: exact / deterministic identity and deduplication.

Leverage ML and LLMs

Probabilistic & Semantic Inference: Maps Machine Learning tasks directly to pipeline stages: Classification for **Aggregation**, Clustering for **Alignment**, and Regression/Generation for **Activation**.

Contribution to Framework Objective: Augments deterministic rules with data-driven predictions, enabling the automated inference of functional Kinds comprehensions, state transitions, and context prompts within the dynamic user API.

Aggregation — classification
Predict Resource/Occurrence Kind membership from observed attributes and relationships.

Alignment — clustering / similarity
Identify potentially equivalent or analogous entities, schemas, dimensions and relationships.

Activation — prediction/regression
Estimate or infer state transitions or interaction outcomes from contextual information.

Leverage MCP (Model Context Protocol)

Dynamic Protocol Binding: Integrates MCP to expose context-aware tool endpoints and AI agent interfaces.

Contribution to Framework Objective: Allows external AI agents and LLMs to dynamically inspect, discover, and trigger activated domain contexts, roles, and executable behaviors exposed by the framework's API.

Purpose

Expose semantic resources, contexts, interactions and augmentation capabilities as discoverable tool/context interfaces.

End-to-end contribution

Aggregation: expose discovered resources and their semantic descriptions.
Alignment: expose matching/query/inference capabilities as tools.
Activation: expose executable Contexts and Interactions as tool invocations.

Leverage FCA

Formal Concept Analysis for classification, types and attributes inference.

Lattice-Based Schema Inference: Constructs formal contexts across three perspectives (*Predicate-as-Context*, *Subject-as-Context*, and *Object-as-Context*) to build concept lattices.

Contribution to Framework Objective: Automates structural classification and type inference in **Aggregation**, while materializing relational schemas and upper-level ontology structures in **Alignment** directly from observed statement patterns.

Purpose

Derive concepts, types, attributes, shared properties and relational schemas from the Base Model.

End-to-end contribution

Aggregation

- infer SubjectKinds, PredicateKinds, ObjectKinds,
- identify common attribute/value structures.

Alignment

- merge formal contexts,
- discover shared conceptual structures,
- derive relational schemas,
- establish higher-level mappings.

Activation

- use inferred concepts and relations as the semantic vocabulary for contexts, roles and interactions.

Materialization

The important architectural step is:

FCA observation → formal concept → inferred Kind/schema → Statement materialization → downstream pipeline

Subjects as Contexts:
Objects: Statement Predicates
Attributes: Statement Objects

Predicates as Contexts:
Objects: Statement Subjects
Attributes: Statement Objects

Objects as Contexts:
Objects: Statement Subjects
Attributes: Statement Predicates

Formal Concept Analysis (FCA) can be used as an inference model over the Base Model to derive classifications, types, attributes and shared properties from the Resource/Occurrence/Statement structures. FCA should operate on the graph structures exposed by the TMRM layer, treating the reified Statements as the source of object–attribute incidence relations rather than introducing a separate representation.

The three FCA contexts defined above provide complementary inference views:

- **Subject context:** Subjects are FCA contexts, Statement Predicates are objects, and Statement Objects are attributes. This allows the system to infer concepts representing groups of Subjects sharing predicate/object characteristics.
- **Predicate context:** Predicates are contexts, Statement Subjects are objects, and Statement Objects are attributes. This can infer common domains, ranges and behavioral/property patterns for predicates.
- **Object context:** Objects are contexts, Statement Subjects are objects, and Statement Predicates are attributes. This can infer classifications of objects based on the predicates through which they occur.

FCA-based Relational Schema Inference

The system can infer relational schemas (rules or "upper concepts") from the structure of the data itself using FCA.

- **FCA Contexts for Relational Analysis:** We use three types of FCA contexts to analyze relationships from different perspectives:
  1. **Predicate-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statement Objects, FCA Context: Statement Predicate). This context reveals which types of subjects relate to which types of objects for a given predicate.
  2. **Subject-as-Context:** (FCA Objects: Statement Predicates, FCA Attributes: Statement Objects, FCA Context: Statement Subjects). This reveals all the relationships and objects associated with a given subject, defining its role.

3. **Object-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statements Predicates, FCA Context: Statement Objects). This reveals all the subjects and actions that affect a given object.

- **Algorithm: Inferring Relational Schema:**
  1. **Select Context:** For a given predicate P (e.g., :worksOn), the Alignment Service constructs the Predicate-as-Context.
  2. **Build Lattice:** It uses an FCA library (e.g., fcalib) to compute the concept lattice from this context.
  3. **Identify Formal Concepts:** Each node in the lattice is a formal concept (A, B), where A is a set of subjects (the "extent") and B is the set of objects they all share (the "intent").
  4. Materialize Schema: Each formal concept represents an inferred relational schema or "upper concept". The system creates a new RDF class for this concept. For a concept where the extent is {dev1, dev2} (both :Developers) and the intent is {projA, projB} (both :Projects), the system can materialize a schema:
     :DeveloperWorksOnProject a rdfs:Class, :RelationalSchema ;
     :hasDomain :Developer ;
     :hasRange :Project .

Leverage CPPE

Algebraic inference by means of numerical prime numbers IDs embeddings.

Purpose

Encode contextual graph structure numerically using Prime IDs and products.

End-to-end contribution

Aggregation: algebraic type inference.
Alignment: similarity, ratios, GCD / LCM-based structural comparison.
Activation: state-transition/product matching.

CPPE pipeline operators are:

- **Contains** → contextual/subgraph filtering.
- **Intersection** → Aggregation/commonality.
- **Axis Shift** → Alignment/reprojection.

Algebraic Semantic Embeddings: Assigns unique prime number IDs to Resources and calculates triadic products (S, P, O Relational Context Vectors). Uses algebraic operators (GCD for intersection, Prime Ratios for axis shifts, Prime Products for state transitions).

Contribution to Framework Objective: Replaces expensive vector search and graph joins with $O(1)$ CPU-native arithmetic (modulo, GCD, LCM). Enables instant type checking, deterministic similarity matching, and reactive stream filtering across all pipeline stages.

Algebraic Semantic Embeddings (CPPE):

Prime Numbers Identifiers:

Every Resource is assigned a unique Prime Number ID (sequence) associated with its URI (via Registry Helper Service API).

Prime ID Embeddings:

Each Resource is assigned an unique incremental Prime Number Identifier.

For a given Resource occurrence in a given Statement context its Prime ID Embedding is calculated as the product of this occurrence Resource Prime ID Embedding with the Prime ID Embeddings of the other two parts of the Statement occurrences.

For example: given an object occurrences its Prime ID Embedding is the product of its Prime ID (Embedding) by the Prime ID (Embedding) of the other occurrences components.

Augmentation Layers. Stream Pipelines:
Aggregation, Alignment, Activation steps. Leverage Prime ID Embeddings for reactive functional composition.

CPPE. FCA-based Embeddings: A Deterministic Approach:

Resource Embeddings are calculated by the product of the Resource's Prime ID by the product of the Resource's Occurrences Attributes and Values Prime IDs.

We will replace LLM-based embeddings with deterministic, structural embeddings derived from FCA contexts and prime number products. This provides explainable similarity based on shared roles and relationships.

- **Contextual Prime Product Embedding (CPPE):** For any Occurrence (i.e., a resource in a specific statement), we can calculate an embedding based on its relational context.
    1. **Define FCA Contexts:** For a given relation (predicate), we can form an FCA context. Example: For the predicate :worksFor:
        - **Objects (G):** The set of all subjects of :worksFor statements (e.g., {id:Alice, id:Bob}).
        - **Attributes (M):** The set of all objects of :worksFor statements (e.g., {id:Google, id:StartupX}).
    2. **Calculate Prime Product:** The CPPE for id:Google within the :worksFor context is the product of the primeIDs of all employees who work there. CPPE(Google, worksFor) = primeID(Alice) * primeID(Bob) * ...
- **Similarity Calculation & Inference:**
    - **Similarity:** The similarity between two entities in the same context is the Greatest Common Divisor (GCD) of their CPPEs. GCD(CPPE(Google), CPPE(StartupX)) reveals the primeID product of their shared employees, giving a measure of personnel overlap.
    - **Relational Inference:** We can infer complex relationships. Consider the goal of finding an "uncle".
        1. Calculate the CPPE for "Person A" in the :brotherOf context (the product of

their siblings' primes).
2. Calculate the CPPE for "Person B" in the :fatherOf context (the product of their children's primes).
3. If GCD(CPPE_brotherOf(A), CPPE_fatherOf(B)) > 1, it means A is the brother of B's father. The system can then materialize a new triple: (A, :uncleOf, ChildOfB). This inference is stored and queryable.

Relational Context Vectors

The core of this approach is the Relational Context Vector (RCV). For any given statement (a reified triple), we compute a vector of three BigInteger values, (S, P, O). Each component is a CPPE calculated from one of the three FCA context perspectives, providing a holistic numerical signature of the statement's role in the graph.

- **RCV Definition:** RCV(statement) = (S, P, O)
  - **S (Subject Context Embedding):** The CPPE of the statement's subject from the Subject-as-Context perspective. This number encodes everything the subject does. S = calculateCPPE(statement.subject, SubjectAsContext)
  - **P (Predicate Context Embedding):** The CPPE of the statement's predicate from the Predicate-as-Context perspective. This number encodes every subject-object pair the predicate connects. P = calculateCPPE(statement.predicate, PredicateAsContext)
  - **O (Object Context Embedding):** The CPPE of the statement's object from the Object-as-Context perspective. This number encodes everything that happens to the object. O = calculateCPPE(statement.object, ObjectAsContext)
- **Implementation:** A Java record RCV(BigInteger s, BigInteger p, BigInteger o). The Index Service is responsible for calculating and caching the RCV for every reified statement in the graph.

Schema Archetypes:

This dual representation is key to performing inference.

- **Instance RCV:** The RCV calculated for a specific, concrete statement (e.g., stmt_123: (dev:Alice, :worksOn, proj:Orion)) is its unique numerical signature. It represents a single data point.
- **Schema RCV (Archetype):** The RCV for a relational schema (e.g., the :DeveloperWorksOnProject schema) is an "archetype" vector. It is calculated by finding the Least Common Multiple (LCM) of the corresponding components of all instance RCVs that belong to that schema.
  - **Algorithm: calculateSchemaRCV(schemaURI)**
    1. Find all instance statements s_i where s_i rdf:type schemaURI.
    2. For each instance s_i, retrieve its cached RCV_i = (S_i, P_i, O_i).
    3. Calculate the schema RCV components:
       - S_schema = LCM(S_1, S_2, ..., S_n)
       - P_schema = LCM(P_1, P_2, ..., P_n)
       - O_schema = LCM(O_1, O_2, ..., O_n)
    4. The result (S_schema, P_schema, O_schema) is the numerical archetype for

the schema. The LCM ensures that the schema's numerical signature is "divisible" by all of its instances.

Subsumption:

Subsumption / Instance Checking (rdf:type):

- **Concept:** An instance belongs to a schema if the instance's RCV "divides into" the schema's RCV.
- **Algorithm: isInstanceOf(instanceRCV, schemaRCV)**
  1. Perform a component-wise modulo operation.
  2. boolean isS = schemaRCV.s.mod(instanceRCV.s).equals(BigInteger.ZERO);
  3. boolean isP = schemaRCV.p.mod(instanceRCV.p).equals(BigInteger.ZERO);
  4. boolean isO = schemaRCV.o.mod(instanceRCV.o).equals(BigInteger.ZERO);
  5. Return isS && isP && isO.
- **Use Case:** This is a high-speed, purely numerical method for checking type constraints, which can be performed in memory without a complex graph query.

Property Chains

Define composed relations (e.g., knowsLanguage) and leverage reasoners for closure. This section details the specific numerical algorithm for the (:Developer)-[:worksOn]->(:Project) and (:Project)-[:usesLanguage]->(:Language) ==> (:Developer)-[:knowsLanguage]->(:Language) inference.

- Step 1: Define the Composition Operator
  compose(RCV1, RCV2)
  - **Inferred Subject (S_inferred):** S_inferred = RCV1.s * RCV2.o
  - **Inferred Object (O_inferred):** O_inferred = RCV1.s * RCV2.o
  - **Inferred Predicate (P_inferred):** P_inferred = RCV1.p * RCV2.p
- **Step 2: Calculate Schema Archetypes**
  - The Index Service calculates archetypal RCVs for source schemas:
    - RCV_worksOn_schema = (S_wo, P_wo, O_wo)
    - RCV_usesLang_schema = (S_ul, P_ul, O_ul)
  - It then calculates the archetypal RCV for the inferred schema (knowsLanguage):
    - S_kl = S_wo * O_ul
    - P_kl = P_wo * P_ul
    - O_kl = S_wo * O_ul
  - This resulting RCV_knowsLang_schema = (S_kl, P_kl, O_kl) is stored.
- Step 3: The Inference Algorithm at Query Time
  A user asks: "Does dev:Alice know lang:Java?"
  1. **Retrieve Instance RCVs:** Retrieve RCV1 for (dev:Alice, :worksOn, proj:Orion) and RCV2 for (proj:Orion, :usesLanguage, lang:Java).
  2. **Calculate Hypothetical Instance RCV:** RCV_hypothetical = compose(RCV1, RCV2).
  3. **Retrieve Schema Archetype:** Retrieve RCV_knowsLang_schema.
  4. **Perform Numerical Check:** boolean knows = isInstanceOf(RCV_hypothetical, RCV_knowsLang_schema).

5. **Result:** If knows is true, the inference is validated.

Querying and Traversal by Numerical Properties:

- **Find by Relational Role:** "Find all entities that have acted as a :Developer".
  - The query becomes: "Find all statements whose instanceRCV.s component divides RCV_dev_schema.s."
- **Traversal by Numerical Similarity:**
  - Start at stmt_A with RCV_A.
  - The next step: "Find stmt_B whose RCV_B has the highest GCD with RCV_A."
  - Allows traversal based on numerically similar relational contexts.

Graph Pipeline Specification: CPPE Algebra for Pipeline Stream Actions:

The **Contextual Prime Product Embedding (CPPE)** algebra treats every node in the graph not as a string or a pointer, but as a unique prime number. By leveraging the **Fundamental Theorem of Arithmetic**, relationships and sets are represented as products. This allows the Reactive Stream Pipeline to perform complex graph "joins" and "path traversals" using simple CPU-native arithmetic.

1. Algebraic Definitions

Let $\mathcal{P}$ be the set of unique Prime IDs assigned to Resource URIs.

For any ContextPoint x:

- $\mathrm{id}(x) \in \mathcal{P}$ (The singleton identity).
- $\mathrm{emb}(x)$ (The cumulative embedding).

The Prime Product Rule:

The embedding of a triadic occurrence (c, o, a) —where c is Context, o is Object, and a is Attribute—is defined as the product of their respective recursive embeddings:

$$\mathrm{emb}(c, o, a) = \mathrm{emb}(c) \times \mathrm{emb}(o) \times \mathrm{emb}(a)$$

Because every factor is a prime (or a product of primes), the result is a unique "coordinate" in the integer space that encodes the entire lineage of the relationship.

2. Pipeline Stream Operators (The CPPE Monad)

In a reactive stream (e.g., Flux<FCAContextPoint>), the following algebraic operators are

used for "Stream Action":

A. The "Contains" Operator (Sub-graph Filtering):

To check if a stream of Objects belongs to a certain Context C or possesses an Attribute A:

- **Logic**: $x \in \text{Context}(C) \iff \text{emb}(x) \pmod{\text{id}(C)} \equiv 0$
- **Reactive Implementation**:
  // Filter objects that are part of the 'Accounting' context
  objectStream.filter(obj ->
  obj.getPrimeIDEmbedding().remainder(accountingID).equals(BigInteger.ZERO));

B. The "Intersection" Operator (Aggregation):

To find commonalities between two streams (finding the "Type" or "Concept"):

- **Logic**: $\text{Type}(x, y) = \gcd(\text{emb}(x), \text{emb}(y))$
- **Stream Action**: The Greatest Common Divisor (GCD) of the embeddings of two nodes represents their shared FCA attributes/contexts.
- **Aggregation Use Case**: When the stream encounters multiple SPO triples, it calculates the "running GCD" of the attributes to infer the Schema (the most specific common super-type).

C. The "Axis Shift" Operator (Alignment):

Alignment predicts a value for a new context by calculating the "Ratio" of change between existing embeddings.

- **Logic**: $\text{Shift}(x, \Delta) = \text{emb}(x) \times \frac{\text{id}(\text{NewContext})}{\text{id}(\text{OldContext})}$
- **Stream Action**: By dividing by the "Old Axis" prime and multiplying by the "New Axis" prime, we re-project the object into a new coordinate.

3. Pipeline Stages Revisited via CPPE:

3.1 Aggregation (Algebraic Type Inference):

The Aggregation pipeline consumes raw SPO streams and produces **"Type Tensors"**.

1. **Input**: Stream of rotated triples (S, P, O).
2. **Action**: For all x sharing attribute a, calculate the product $\prod \text{id}(x)$.
3. **Inference**: If $\text{emb}(x)$ is divisible by the product of a set of attribute primes, x is

classified into that Type.
4. **Complexity**: $O(1)$ lookup via modulo instead of $O(N)$ graph search.

3.2 Alignment (Vector/Prime Space Prediction):

Alignment uses the **Prime Ratio** to detect structural similarity.

1. **Input**: Aggregated Types.
2. **Action**: Map the distance between objects O_1 and O_2 as $d = \frac{\text{emb}(O_1)}{\text{emb}(O_2)}$.
3. **Discovery**: If O_3 and O_4 share the same ratio d, the system aligns them as being "analogous" in different contexts (e.g., "CEO is to Company A what Principal is to School B").

3.3 Activation (State Transition Product):

Activation is the "Activation Energy" required to move from one prime state to another.

- **Transition Formula**: $\text{Transition} = \text{emb}(\text{Prev}) \otimes \text{emb}(\text{Next})$
- **Monadic Wrap**: The Occurrence Monad captures the prime state of an actor. When the stream processes a "Role" event, it multiplies the actor's prime ID by the role's prime ID.
- **Trigger**: If the resulting product matches a known "Success Pattern" (a pre-calculated product of requirements), the **Activation** is fired (e.g., triggering a webhook or a UI change).

Leverage W3C DIDs (Distributed Resource Identifiers)

Decentralized Resource Identity: Uses DIDs and DID Documents as a globally addressable registry mechanism for resources, schemas, Kinds, and service endpoints.

Contribution to Framework Objective: Establishes cross-boundary identity, provenance tracking, and capability discovery. Ensures distributed domain services can publish, verify, and synchronize graph state and interaction boundaries securely across administrative boundaries.

Use W3C Decentralized Identifiers (DIDs) as a **distributed semantic registry mechanism** for Resources, Statements, Kinds, Schemas, Contexts and other semantic artifacts participating in the service. A DID provides a stable, globally addressable identifier whose associated DID Document can describe the identity, endpoints, metadata and verification mechanisms required to resolve and interact with the corresponding semantic resource.

Within the proposed architecture, the DID layer acts primarily as a **distributed identity and registry service**, while the TMRM/Base Model remains responsible for representing the

semantic graph and its Resource/Occurrence/Statement structures. DIDs can therefore provide a common identity mechanism across independently managed data sources, semantic services and augmentation agents.

The Registry Service could expose APIs/functions such as:

- **Resource registration:** register a semantic Resource and associate it with a DID and its Base Model URI/identifier.
- **DID resolution:** resolve a DID to its current metadata, semantic endpoints and associated resource information.
- **Resource discovery:** discover semantic resources, schemas, Kinds, contexts or services by DID, type, capability or semantic relationship.
- **Identity mapping:** maintain mappings between external identifiers, source-specific URIs and canonical DIDs.
- **Document/version management:** publish and retrieve changes to the metadata describing a registered resource, supporting versioned semantic definitions.
- **Verification:** verify the provenance, controller and integrity of registered semantic resources or statements where appropriate.
- **Service endpoint discovery:** expose the APIs, messaging endpoints or graph services through which a registered semantic resource can be accessed.
- **Delegation/authorization:** provide a foundation for determining which agents or services are allowed to publish, modify or operate on registered semantic resources.
- **Provenance and lineage:** associate resources and semantic statements with their originating registries, sources, agents or augmentation processes.
- **Distributed synchronization:** propagate registry changes through the existing Event Bus so that local indexes and semantic services can react to resource registration, update or removal events.

A DID can consequently serve as the **distributed identity coordinate** for a semantic resource, while the Registry Service maintains the relationship between that identity and the actual semantic graph structures, endpoints and metadata. The Index Service can use these identifiers when maintaining searchable and numerical indexes, including the Prime ID mappings used by CPPE.

This separation allows the architecture to distinguish between **identity, registration and semantic representation**: DIDs identify and locate resources across administrative boundaries; the Registry Service manages their distributed registration and discovery; and the TMRM/Base Model represents their semantic relationships. Augmentation Pipeline agents can then consume resources discovered through the registry and produce Property, Rule and Context Statements as part of the existing Aggregation, Alignment and Activation pipeline.

The resulting principle

I would therefore define the architecture as:

> **DIDs identify distributed domain services and advertise their semantic business capabilities. Domain APIs provide the invocation boundary for those capabilities. The semantic graph provides the shared representation**

of the domain entities and their state. Aggregation, Alignment and Activation determine how a requested business interaction maps onto that semantic graph, while deterministic semantic primitives perform the resulting state changes.

This is arguably a better match for the document than a purely resource-oriented DID API because the document ultimately aims at **contexts, interactions and executable behaviors**, not merely distributed CRUD over resources.

Techniques Summary

Technique	Primary role	Aggregation	Alignment	Activation	Main implementation boundary
TMRM	Semantic graph/metamodel	Resource/Kind structure	Identity/context merge	Semantic substrate	Graph model
Dimensional features	Axes/order/state	Type/state hierarchy	Ordering/analogy	State transitions	Semantic structure
Formal grammars	Rules/execution	Infer productions	Match/order productions	Execute productions	Rule model
RDF/RDFS/OWL/SPARQL	Semantic-web infrastructure	Query/type	Graph/ontology matching	Query/materialize	Graph tooling
Sets	Algebraic primitives	Intersection/grouping	Inclusion/difference	Candidate sets	Core algorithms
ML/LLMs	Learned inference	Classification	Clustering	Prediction	Inference helpers
MCP	Tool/interaction boundary	Discovery	Semantic tools	Invocation	External interface
FCA	Conceptual inference	Classification	Concept/schema matching	Context vocabulary	Inference engine
CPPE	Deterministic numerical inference	Algebraic typing	Similarity/alignment	Transition matching	Index/algebra
DIDs	Distributed identity/registry	Source discovery	Identity mapping	Service discovery	Distributed infrastructure

Implementation techniques are interchangeable inference and execution mechanisms around a common semantic core. They do not define separate models; they operate on the

same Resources, Occurrences, Statements and Kinds, contribute specialized computations to Aggregation, Alignment and Activation, and materialize their results back into the same statement-stream architecture.

Appendix: Enterprise Scenario: Cross-System Order-to-Delivery Integration

Consider a global manufacturing enterprise operating four distinct systems:

- **CRM (Salesforce):** Manages customer orders and account leads.
- **ERP (SAP S/4HANA):** Handles billing, general ledger, and invoicing.
- **SCM / WMS (Manhattan Associates):** Tracks warehouse stock, picking, and shipment logistics.
- **Project Management (Jira / Asana):** Tracks customer onboarding and post-sale hardware installation projects.

Without a centralized integration model, bridging these systems requires custom point-to-point ETL pipelines or rigid middleware APIs. Below is a concrete walk-through demonstrating how the **Semantic Streams Augmentation Pipeline** acts as an intra-/inter-application integration facade.

1. Raw Source Data Streams (Datasource Ingestion)

The integration facade consumes real-time change-data-capture (CDC) events exported into raw RDF Context-Subject-Predicate-Object (CSPO) Quad Statements across all systems:

CRM Event Stream (Salesforce)
(:ctx_CRM, :Customer_ACME, :placesOrder, :Order_1001)

ERP Event Stream (SAP)
(:ctx_ERP, :Order_1001, :generatesInvoice, :Invoice_5001)

SCM Event Stream (Manhattan WMS)
(:ctx_SCM, :Order_1001, :dispatchedIn, :Shipment_9001)

Project Management Event Stream (Jira)
(:ctx_PM, :Order_1001, :triggersProject, :Project_7001)

Every entity is assigned a unique Prime Number ID via the Registry Service to enable algebraic graph transformations (CPPE):

- $primeID(:Customer_ACME) = 3$
- $primeID(:placesOrder) = 5$
- $primeID(:Order_1001) = 7$

- $primeID(:generatesInvoice) = 11$
- $primeID(:Invoice_5001) = 13$

2. Stage 1: Aggregation Service Agent (Data Layer - Type & Role Inference)

Processing & Inference

The **Aggregation Agent** consumes raw RDF quads from the messaging stream and performs structural type and role inference using Sets intersections, Formal Concept Analysis (FCA), and Contextual Prime Product Embeddings (CPPE).

By analyzing shared attributes and predicates across the raw streams, it infers the underlying **Kinds** (extensional type definitions):

1. **SubjectKind (SK):** Groups entities playing a subject role with shared predicates (e.g., :Customer_ACME as a CustomerSubjectKind).
2. **PredicateKind (PK):** Grouping of relationship attributes (e.g., :placesOrder and :generatesInvoice aggregated into functional property types).
3. **ObjectKind (OK):** Grouping of target value types (e.g., :Order_1001 acting as an OrderObjectKind in CRM and OrderSubjectKind in ERP).
4. **ContextKind (CK):** Aggregates overall situational contexts (e.g., SalesOrderContextKind).

Raw SPO Quads ⟶ [Aggregation Agent: Fold] ⟶ Property Quad Statements (Kinds)

Production Output (Property Quad Statements)

The agent folds raw statements into **Property Quad Statements** and publishes them to the aggregationProductionTopic:

Format: (ContextCK, TypeSK, PropertyPK, ValueOK)
(:SalesOrderContextKind, :CustomerSubjectKind, :OrderPlacementPredicateKind, :OrderObjectKind)
(:BillingContextKind, :OrderSubjectKind, :InvoicingPredicateKind, :InvoiceObjectKind)

3. Stage 2: Alignment Service Agent (Information Layer - Schema Matching & Identity Resolution)

Processing & Inference

The **Alignment Agent** consumes Property Quad Statements from the Aggregation Layer. Its primary goals are **identity resolution**, **link completion**, and **n-ary dimensional ordering**:

1. **Identity Resolution via CPPE / TMRM:** The agent calculates the Greatest Common Divisor (GCD) of CPPE prime embeddings and inspects ISO TMRM subject proxies across streams. It recognizes that :Order_1001 in CRM, ERP, SCM, and PM represents the exact same real-world entity, merging these disparate topic proxies into a unified subject.
2. **Dimensional Axis & State Transition Modeling:** It maps sequential business flow transitions (Previous, Current, Next - PCN) across dimensions:
   $$\text{Order Placed (CRM)} \longrightarrow \text{Invoiced (ERP)} \longrightarrow \text{Shipped (SCM)} \longrightarrow \text{Onboarding Started (PM)}$$

Property Statements ⟶ [Alignment Agent: Merge & Align] ⟶ Rule Quad Statements

Production Output (Rule Quad Statements)

The agent synthesizes cross-system data into an N-ary reified relationship rule and publishes it to the alignmentProductionTopic:

Format: (RelationshipCK, SourceEndSK, MeasurementContextPK, DestinationEndOK)
(:OrderToFulfillmentRelationshipCK, :CustomerOrderRoleSK, :FulfillmentTrackingPK, :ProjectOnboardingRoleOK)

This single **Rule Statement** links the origin customer order in Salesforce to the active deployment project in Jira across accounting and warehouse steps.

4. Stage 3: Activation Service Agent (Knowledge Layer - Use Cases & Behaviors)

Processing & Inference

The **Activation Agent** translates structural rules into executable **DCI (Data, Context, Interaction)** patterns. It identifies active business actors, roles, and executable state transitions:

- **DCI Context (CK):** :CustomerOnboardingDCIContext
- **Roles (SK / OK):** :AccountManagerRole, :ProjectManagerRole, :SystemIntegratorActor
- **Interactions (OK):** :TriggerOnboardingProject, :SyncBillingWithShipment

Rule Statements ⟶ [Activation Agent: Activate & Map] ⟶ Context Quad Statements

Production Output (Executable Context Quad Statements)

Format: (DCIContextCK, RoleSK, ActorPK, InteractionOK)

(:CustomerOnboardingDCIContextCK, :ProjectManagerRoleSK, :JiraSyncActorPK, :InitiateOnboardingInteractionOK)

5. Unified Contexts & Interactions API (Dynamic Endpoint Execution)

The framework exposes a dynamic, discoverable API (such as HAL or Model Context Protocol / MCP) based on the activated knowledge statements. External systems or AI agents interact with this unified facade rather than querying individual backends.

Architectural Summary Table

Phase / Layer	**Input Statement Type**	**Primary Mechanics**	**Output Statement Type**	**Backend Impact**
Ingestion	Native DB Events / CDC	RDF Quad Serialization	Raw CSPO Quads	Monitors CRM, ERP, SCM, PM.
1. Aggregation	Raw CSPO Quads	Sets / FCA Lattices / CPPE Modulo ($O(1)$)	Property Quad Statements	Classifies raw entities into Kinds.
2. Alignment	Property Quad Statements	CPPE GCD/LCM, TMRM Identity Merge, PCN Ordering	Rule Quad Statements	Unifies :Order_1001 across systems.
3. Activation	Rule Quad Statements	DCI Context Mapping, Grammar Productions	Context Quad Statements	Exposes cross-system workflows.
API / Execution	Context Quad Statements	Dynamic HAL / MCP Endpoints	Synchronized Backend Actions	Updates SAP, Jira, Manhattan state.

Execution Flow Example

1. **Triggering an Interaction:** An Account Manager invokes the :InitiateOnboardingInteractionOK endpoint via the dynamic API.
2. **State Transition Propagation (Homoiconic Unfold):**
    - The pipeline unfolds the Context Statement back into its underlying Rule and Property Statements.
    - The state transition product formula evaluates: $State_{new} = State_{current} \times primeID(:InitiateOnboarding)$
3. **Backend Synchronization:**
    - **SCM (Manhattan):** Updates shipment status to DELIVERED.
    - **ERP (SAP):** Updates invoice status from PENDING_DELIVERY to READY_TO_BILL.
    - **PM (Jira):** Creates and assigns the customer onboarding project sprint automatically.

The underlying enterprise applications remain in sync with the integration facade's state flows, operating continuously while the framework provides a single, unified interaction view for all cross-application workflows.

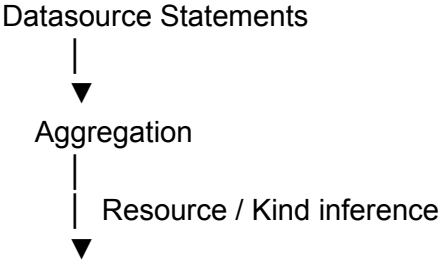
Appendix: Use Case Discovery and Unified Facade

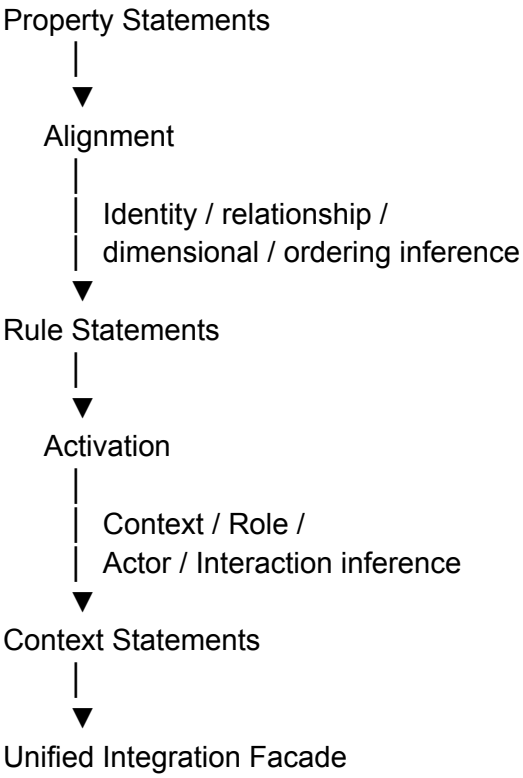
The objective of the Semantic Streams Augmentation framework is not only to integrate heterogeneous datasource models, but to discover higher-level **Contexts and Interactions** that can be exposed as a unified facade over the integrated applications. The source applications remain operational and are synchronized with inferred knowledge and state changes, while the framework provides an integrated semantic and interaction view across their boundaries.

A **Use Case** can therefore be regarded as an inferred `ContextKind` whose participating Resources acquire application-specific `Role`, `Actor` and `Interaction` meanings. Use Case discovery is the process of deriving these higher-level structures from the Property and Rule Statements produced by the Aggregation and Alignment layers.

Use Case Discovery Pipeline

The discovery process follows the same three-layer Augmentation Pipeline:





This corresponds to the existing Agent model in which Aggregation consumes raw RDF CSPO Statements and produces Property Statements, Alignment consumes those Property Statements and produces Rule Statements, and Activation consumes the aligned information and produces Context Statements.

The key distinction is that the output of the pipeline is not necessarily another representation of one datasource. It can be a **new, inferred interaction model spanning several datasources and applications**.

Intra-Application Use Case Discovery

An intra-application Use Case is inferred when resources belonging to different subsystems of the same application can be connected through shared identities, Kinds, relationships and state transitions.

For example, an enterprise application may contain independent:

Human Resources
Payroll
Directory
Access Control

datasources.

Their source Statements may independently describe:

HR.Employee
Payroll.EmployeeAccount

Directory.Person
AccessControl.Role

Although their source schemas differ, Aggregation can identify recurring Resource Occurrences and infer Kinds such as:

EmployeeSubjectKind
DepartmentObjectKind
EmploymentRelationshipPredicateKind
EmployeeRoleSubjectKind

The current Aggregation model already demonstrates this type of inference for employment, transforming raw `(Context, Subject, Predicate, Object)` Statements into Context, Subject, Predicate and Object Kinds and then into a higher-level Property Statement.

Alignment can subsequently infer relationships between the independently modeled entities:

HR.Employee
  │
  ├── Directory.Person
  │
  ├── Payroll.EmployeeAccount
  │
  └── AccessControl.EmployeeRole

The resulting semantic graph can therefore represent a higher-level concept such as:

Employee
  ├── identity
  ├── employment
  ├── compensation
  └── accessRole

without requiring the underlying applications to share their internal data models.

Activation can then recognize a Context such as:

EmployeeLifecycle

with Roles and Interactions such as:

Roles:
  Employee
  HRAdministrator
  PayrollSystem
  AccessControlSystem

Interactions:
  createEmployee

assignRole
updateCompensation
activateAccess
terminateEmployee

The Activation layer is explicitly modeled around Contexts, Roles, Actors and Interactions.

The resulting facade can consequently expose an interaction-oriented API:

```
GET  /employees/{id}
GET  /employees/{id}/employment
GET  /employees/{id}/role
POST /employees/{id}/change-role
POST /employees/{id}/terminate
```

The endpoint does not have to correspond one-to-one with an underlying application endpoint. Its implementation can traverse the inferred semantic structures and invoke the appropriate underlying Resources or application services.

Inter-Application Use Case Discovery

An inter-application Use Case extends the same mechanism across independently managed applications.

For example:

```
CRM
  Customer
  Opportunity

ERP
  Account
  Invoice
  Payment

Support
  Customer
  ServiceCase
```

The applications may use different identifiers, schemas and relationship structures to describe overlapping domain entities.

Aggregation first derives their local Kinds:

```
CRM.CustomerSubjectKind
ERP.AccountSubjectKind
Support.CustomerSubjectKind

OpportunityPredicateKind
```

InvoicePredicateKind
PaymentPredicateKind
ServiceCasePredicateKind

Alignment then attempts to establish equivalence, analogy or relational correspondence between these structures.

A possible result is:

CRM.Customer
    │
    ├──────────────── ERP.Account
    │
    └──────────────── Support.Customer

with additional relationships:

Customer ── hasOpportunity ──> Opportunity
Customer ── hasInvoice ─────────> Invoice
Customer ── hasPayment ─────────> Payment
Customer ── hasCase ──────────────> ServiceCase

The important result is not simply a union of the three source graphs. Alignment produces a semantic structure in which the different source representations become occurrences of a common integrated model.

The proposal explicitly identifies identity resolution and merging as a central requirement, while preserving provenance and contextual information when the same underlying subject occurs in multiple streams.

Activation can then derive a higher-level:

CustomerServiceContext

with:

Roles:
  Customer
  SalesRepresentative
  BillingAgent
  SupportAgent

Actors:
  CRM
  ERP
  Support

Interactions:
  viewCustomer
  createOpportunity

issueInvoice
recordPayment
openServiceCase

The unified facade could consequently expose:

```
GET  /customers/{id}
GET  /customers/{id}/opportunities
GET  /customers/{id}/billing
GET  /customers/{id}/service-cases
POST /customers/{id}/open-service-case
```

The important property is that these endpoints are derived from the **integrated semantic Context**, rather than being a static aggregation of three application APIs.

Use Case as an Inferred Semantic Contract

A discovered Use Case can be represented as a semantic contract consisting of:

```
Context
  Roles
    Actors
      Interactions
        Preconditions
        State
        Results
        Transitions
```

The Base Model provides the primitive structure from which this contract can be represented using Resources, Occurrences and Statements. Kinds then classify recurring structures, while Rule Statements express the relationships that connect them.

For example:

```
(CustomerOnboardingContext,
 CustomerRole,
 CreateCustomerInteraction,
 Customer)
```

can represent a contextual interaction rather than merely an entity relationship.

This follows the document's progression from raw CSPO Statements, through Kinds and Property Statements, to Rule Statements and finally Context Statements.

Use Case Discovery from Dimensional and State Structures

Use Case discovery can also exploit the dimensional features proposed in the model.

The specification describes relationships in terms of axes, ordering and Previous/Current/Next state structures, for example:

(Unemployed, Employment, Employee)

(Junior, Promotion, Senior)

(Single, Marriage, Married)
(Married, Divorce, Divorced)

These structures can be interpreted as generalized state-transition patterns.

Consequently, Activation can discover not only static contexts but **behavioral contexts**.

For example:

EmployeeOnboarding

Previous:
  Applicant

Interaction:
  Hire

Current:
  Employee

Next:
  ActiveEmployee

or:

OrderFulfillment

Previous:
  CreatedOrder

Interaction:
  Fulfill

Current:
  FulfilledOrder

Next:
  ShippedOrder

Such patterns allow the facade to expose transitions rather than merely CRUD operations:

POST /orders/{id}/fulfill

POST /orders/{id}/ship
POST /employees/{id}/activate

The interaction is therefore the semantic representation of a state transition across one or more underlying applications.

Composite Interactions

An especially important case is an Interaction whose execution spans several applications.

For example:

CustomerOnboarding

may be inferred as:

createCustomer
  ↓
verifyIdentity
  ↓
evaluateEligibility
  ↓
createAccount
  ↓
activateAccount

where different steps belong to different underlying applications.

The integration facade exposes:

POST /customer-onboarding

while the Activation layer resolves the contextual execution path:

CRM
  └── createCustomer

Identity Service
  └── verifyIdentity

Compliance
  └── evaluateEligibility

ERP
  └── createAccount

Access / Service Platform
  └── activateAccount

The semantic graph therefore becomes the intermediary between **what the integrated system means** and **how its constituent applications implement it**.

This follows the proposal's “code as data” idea: Resources, Statements and Kinds can be applied as templates, transformations and functional compositions, with merge, fold and unfold providing the stream-processing mechanism.

Unified Read Facade and Unified Interaction Facade

It is useful to distinguish two levels of facade.

Unified Resource Facade

Provides integrated views of Resources:

/customer/{id}
/employee/{id}
/order/{id}
/product/{id}

These are primarily graph traversal and information integration operations.

Unified Interaction Facade

Provides contextual operations:

/customer-onboarding
/order-fulfillment
/employee-change-role
/customer-service

These are Activation-level operations whose semantics are defined by Contexts, Roles, Actors, Interactions and state transitions.

This distinction is important because the document's final API concept is explicitly centered on activated Contexts and Interactions, with interfaces rendered over contextual behaviors and role executions.

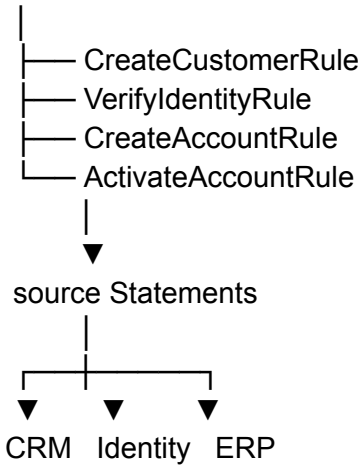
Traceability from Facade Interaction to Datasources

Every inferred facade operation should remain traceable to the Statements and Kinds from which it was derived.

For example:

POST /customer-onboarding

CustomerOnboardingContext

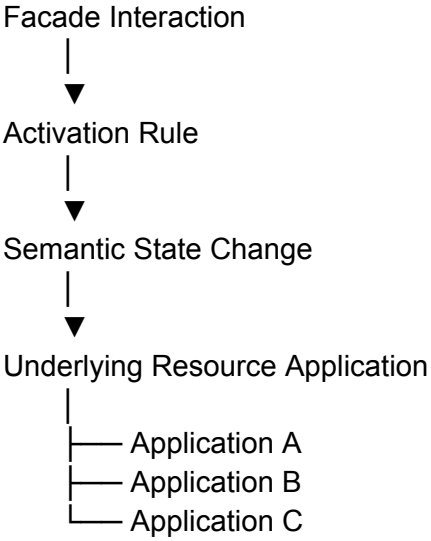


This preserves the central property of the proposal that inferred knowledge is materialized into Statements and propagated through the same stream processing architecture as source knowledge. The pipeline can merge new information with previously known and inferred Statements and then fold or unfold it as required.

The facade is therefore not a black-box orchestration layer. Its operations can be understood as **materialized semantic executions derived from the graph**.

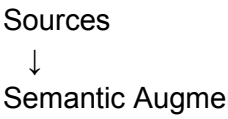
Datasource Synchronization

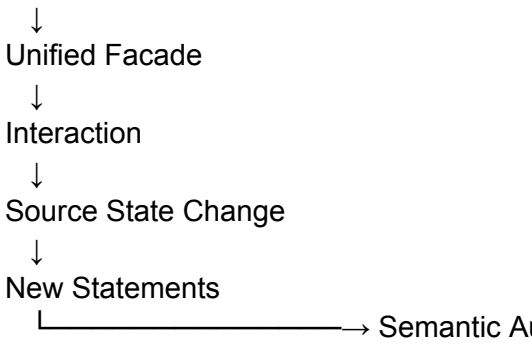
A write through the unified facade can follow the reverse direction:



The resulting source changes generate new Statements, which re-enter the augmentation pipeline and can update the inferred graph.

This creates a feedback loop:

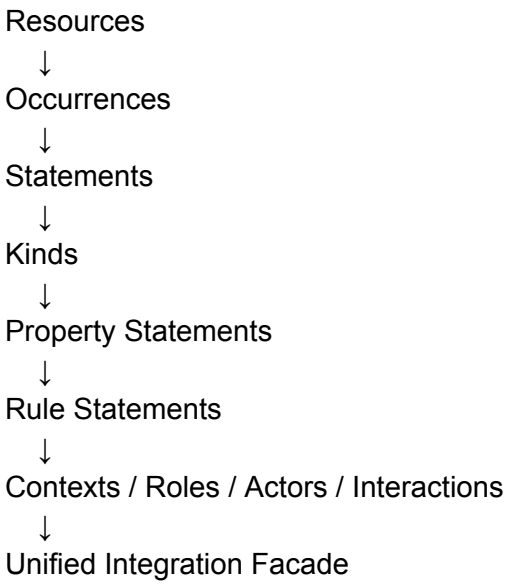




This is consistent with the proposal's stated intention that the underlying applications remain operational while reflecting changes made through the integration mechanisms.

Facade Generation Principle

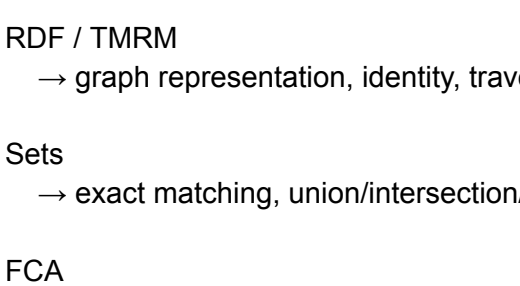
The unified facade can therefore be viewed as a projection of the activated semantic graph:



The facade is **not another independent domain model**. It is an executable projection of the knowledge inferred by the Augmentation Pipeline.

Relationship to Implementation Techniques

The different implementation techniques can contribute at different points in this discovery process:



→ concept, Kind and relational-schema inference

CPPE
→ contextual numerical identity, similarity and traversal

Dimensional Features
→ axes, ordering and state transitions

Formal Grammars
→ executable Context/Interaction productions

ML / LLMs
→ candidate classification, alignment and behavioral inference

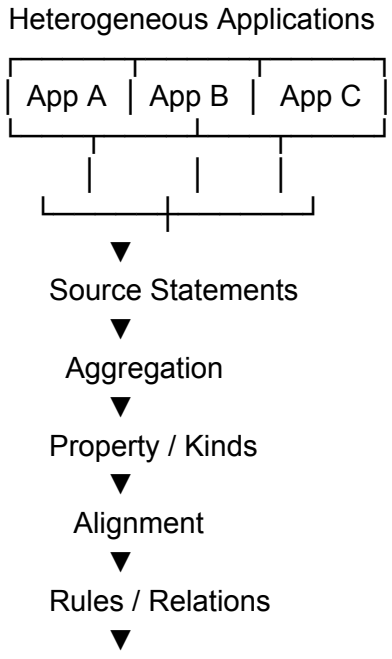
DIDs / Registry
→ distributed Resource and service identity

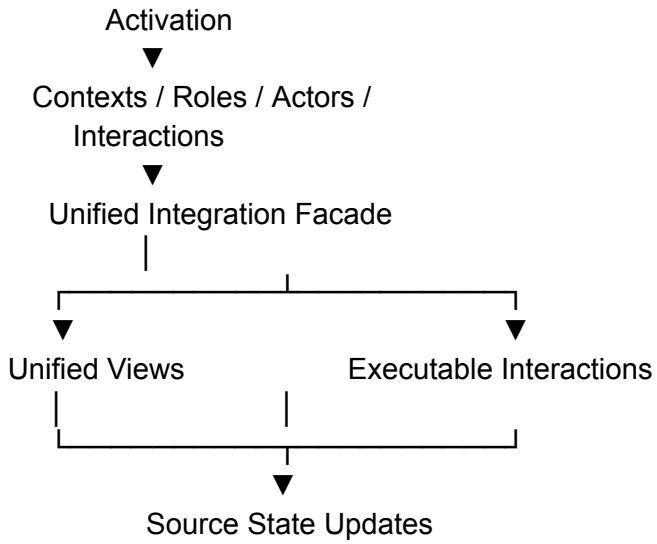
MCP / APIs
→ external discovery and invocation of activated capabilities

All of these mechanisms ultimately converge on the same Base Model and the same Aggregation → Alignment → Activation flow rather than producing independent integration artifacts. The proposal explicitly describes the Helper Services as providing functional APIs over the Base Model and allowing techniques such as FCA, CPPE, RDF inference, Sets and ML/LLMs to participate in a common `Resource::apply` composition model.

Resulting architecture

The overall result can be summarized as:





This gives the proposal a concrete interpretation of its central idea: **the system does not merely integrate application data; it discovers and exposes the interactions that become meaningful when those applications are viewed as one semantic system.**

Intra-application use case: inferred by connecting multiple subsystems/datasources belonging to the same application.

Inter-application use case: inferred by aligning corresponding concepts and relationships across different applications.

The following examples are extensions of the proposal rather than use cases explicitly specified in the document.

1. Intra-application: Employee lifecycle facade

Imagine one organization's HR application has:

`Employee DB` → employees, positions
`Payroll DB` → salary/payroll records
`Directory API` → identities, departments
`Access-control system` → roles and permissions

The proposal could infer that all four are describing overlapping Resources and occurrences of the same employee.

What Aggregation discovers

The Aggregation layer could derive Kinds such as:

```
EmployeeSubjectKind
DepartmentObjectKind
EmploymentRelationshipPredicateKind
```

`SalaryMeasurementPredicateKind`
`EmployeeContextKind`

This is very close to the employment example already in the document, where raw `(C,S,P,O)` statements become Context/Subject/Predicate/Object Kinds and then a property statement such as an employment relationship.

What Alignment discovers

It could then recognize structures such as:

`HR.employee → Directory.person`

and

`HR.position → AccessControl.role`

and infer relationships such as:

`Employee --hasCurrentRole--> Role`

even when the source systems use different identifiers or intermediate structures.

The document's Alignment layer is explicitly intended for ontology matching, links, attributes, dimensional relationships and ordering.

What Activation exposes

The resulting facade might expose:

GET /employees/{id}
GET /employees/{id}/employment
GET /employees/{id}/current-role
GET /employees/{id}/compensation
POST /employees/{id}/change-role

The important point is that `/employees/{id}/current-role` would not necessarily correspond to one backend endpoint. The facade could derive the answer by traversing the integrated semantic graph and then invoke the appropriate underlying system when a state-changing interaction is requested.

That is consistent with the document's idea of dynamic endpoints over Activation-generated Contexts, Interactions, roles and behaviors.

2. Intra-application: Order fulfillment facade

Consider an application consisting of:

```
Order Management
Inventory
Warehouse
Shipping
```

Internally they might have quite different models:

Order
  contains OrderLine

Inventory
  reserves SKU

Warehouse
  picks Package

Shipping
  creates Shipment

Aggregation

The framework could infer Kinds around:

Customer
Order
OrderLine
Product
InventoryReservation
Package
Shipment

and recurring relationships such as:

Order --contains--> Product
Product --reservedBy--> Order
Order --fulfilledBy--> Package
Package --shippedAs--> Shipment

Alignment

The interesting inference is that these are not independent concepts but stages of one larger relationship:

Order
  → Reservation
  → Fulfillment
  → Shipment

The dimensional machinery in the document is suited to expressing these as ordered states/transitions, including Previous/Current/Next structures.

Activation

The unified facade could expose a business-level interaction:

Order.fulfill()

rather than forcing a client to know that fulfilling the order requires:

Inventory.reserve()
Warehouse.createPick()
Warehouse.confirmPack()
Shipping.createShipment()

In the proposal's terms, the interaction becomes an inferred Context/Role/Actor/Interaction structure, while the underlying services remain separate.

3. Inter-application: Customer 360 facade

Now consider two independent applications:

CRM

Customer
Account
Contact
Opportunity

Billing

Account
Invoice
Payment

Possibly also:

Support

Customer
Case
Ticket

Aggregation

Each datasource independently produces its own Kinds.

For example:

CRM.Customer
Billing.Account
Support.Customer

may have different schemas but overlapping occurrence structures.

Alignment

The Alignment layer could discover a correspondence like:

CRM.Customer
↕
Billing.Account
↕
Support.Customer

and merge those identities into one canonical Resource while retaining provenance/context from each datasource.

Identity and merging are explicitly part of the proposed graph infrastructure.

It could then infer a broader relationship:

Customer
├── hasOpportunity
├── hasInvoice
├── hasPayment
└── hasSupportCase

Unified facade

A client could ask:

GET /customers/{id}

and receive a semantic aggregate assembled from all three applications.

More interestingly:

GET /customers/{id}/360

could be an **Activation-generated Context** rather than a static CRUD resource.

The distinction matters: the facade represents the **integrated context** rather than exposing three application schemas side by side.

4. Inter-application: University ↔ employment ecosystem

This one is especially close to examples already present in the document.

Suppose there are:

University system

Student
Program
Course
Degree

HR system

Employee
Employer
Position

The proposal already gives the conceptual example:

> Employee is to Employer in a WorkRelationship as Student is to School in a StudentshipRelationship.

That appears in the dimensional/alignment discussion.

Aggregation

The framework could derive:

EmployeeSubjectKind
EmployerObjectKind
WorkRelationshipPredicateKind

StudentSubjectKind
SchoolObjectKind
StudentshipRelationshipPredicateKind

Alignment

It could infer an analogy such as:

Employee : Employer
≈
Student : School

That does not mean the two concepts are identical. Rather, their **relational roles are structurally analogous**.

This is exactly the type of dimensional relationship the proposal is trying to materialize through PredicateKinds, axes and comparison encodings.

Activation

That could support an interaction such as:

Student.graduate()
↓
EligibleForEmployment
↓
Employee.onboard()

The university and HR applications could remain independent while the integration facade exposes an organization-level workflow.

5. Inter-application: Product-to-service lifecycle

Imagine:

PLM

Product
Part
Revision

ERP

Product
PurchaseOrder
Supplier

Field Service

InstalledAsset
ServiceTicket
Technician

The semantic graph could discover that:

PLM.ProductRevision
↓
ERP.Product
↓
FieldService.InstalledAsset

represents different contextual occurrences of what is effectively the same domain Resource.

Alignment features

This is where the combination of **FCA + CPPE + dimensional comparison** becomes interesting.

FCA can detect common structural concepts, while CPPE provides numerical contextual signatures for comparison and indexing. The document explicitly proposes FCA for classification/schema inference and CPPE for deterministic contextual similarity and relational inference.

Unified facade

Instead of exposing:

GET /plm/products/...
GET /erp/products/...
GET /field-service/assets/...

the integration layer could expose:

GET /products/{id}
GET /products/{id}/revisions
GET /products/{id}/installed-assets
GET /products/{id}/service-history

The semantic layer determines which underlying datasource supplies each portion.

6. Inter-application: Lead → Customer → Contract → Service

This is a good example of an **end-to-end use case discovered across application boundaries**.

Suppose:

`Marketing` owns Leads
`CRM` owns Customers
`ERP` owns Contracts
`Support` owns Service Cases

The applications do not necessarily share a unified model.

The semantic pipeline could discover:

Lead
↓
Customer
↓
Contract

↓
ServiceRelationship

and potentially a higher-level Context:

CustomerLifecycle

Aggregation

Infer the different entity and relationship Kinds.

Alignment

Resolve:

Marketing.Lead.person
CRM.Customer.person
ERP.Contract.account
Support.Case.account

into a coherent network of Resources.

Activation

Expose interactions such as:

customer.convertLead()
customer.activateContract()
customer.openServiceCase()

The important shift is that the facade operates at the level of **business interactions**, while the underlying applications remain implementation-specific.

7. Inter-application: Finance reconciliation

Consider:

Sales

Order
OrderTotal
Customer

Billing

Invoice
InvoiceTotal
Account

Payments

Payment
Settlement
Account

The framework could infer a common financial process:

Order
  → Invoice
  → Payment
  → Settlement

Alignment

Structural matching could discover:

Sales.Customer
    ≈
Billing.Account
    ≈
Payments.Account

and dimensional relationships between:

OrderTotal
InvoiceTotal
PaymentAmount

Activation

The integration facade could expose:

GET /orders/{id}/financial-status
GET /customers/{id}/outstanding-balance
POST /orders/{id}/settle

The last interaction could activate actions across multiple systems rather than simply reading data.

This fits the proposal particularly well because Activation is intended to move from aligned information into **Contexts, Roles, Actors and Interactions**.

8. A more interesting class: inferred "composite" interactions

The most powerful facade feature, conceptually, may not be CRUD aggregation at all.

Suppose the graph has inferred:

Context:
  CustomerOnboarding

Roles:
  Applicant
  SalesRepresentative
  ComplianceOfficer

Actors:
  Customer
  CRM
  IdentityService
  ComplianceSystem
  ERP

Interactions:
  createCustomer
  verifyIdentity
  evaluateCompliance
  createAccount
  activateAccount

The facade could expose a single contextual interaction:

POST /customer-onboarding

whose implementation traverses the inferred interaction graph.

The proposal's Activation model is explicitly about discovering **Contexts (Use Cases), Roles, Actors and Interactions**, rather than merely aggregating records.

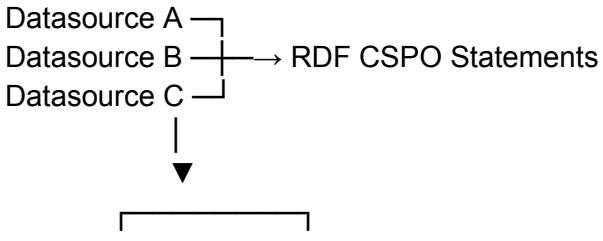
That makes the facade concept much richer:

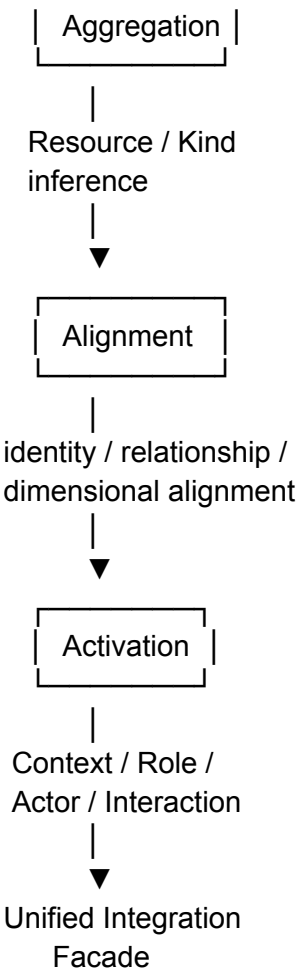
> **Data facade:** “give me the integrated customer.”
> **Interaction facade:** “perform the customer onboarding context.”

The latter is much closer to the stated objective.

How the pipeline would infer these use cases

A useful mental model is:





This corresponds closely to the defined Agent flow:

Raw RDF → Aggregation Property Statements → Alignment Rule Statements → Activation Context Statements.

And because the agents operate through merging, folding and unfolding, an inferred higher-level interaction can potentially be traced back to the lower-level Statements that produced it.

What the unified facade could actually expose

I would group the resulting facade features into five categories.

Facade capability	Example
Unified Resources	`/customers/{id}` combines CRM + Billing + Support
Unified Relationships	`/employees/{id}/current-role` traverses HR + Directory + Access Control
Derived views	`/orders/{id}/fulfillment-status`

Contextual queries	`/customers/{id}/financial-status`
Executable interactions	`POST /customer-onboarding`, `POST /employees/{id}/change-role`

The last category is the architectural payoff: the dynamic API section describes rendering interfaces over **Contexts, Interactions, behaviors, states and role executions**, rather than simply mirroring backend APIs.

Appendix: ChatGPT Conversation

https://chatgpt.com/share/6a2c26c5-eab0-83e9-a20b-f4f5807e8e26

Appendix: Gemini Conversation

https://share.gemini.google/5jyLb507UYYb